CHAPTER 22

Serial Communication Protocols

This chapter introduces four important serial communication protocols, including universal asynchronous receiver and transmitter (UART), inter-integrated circuit (I^2C), serial peripheral interface (SPI), universal serial bus (USB). Serial communication transfers a single bit each time and uses either a single wire for each communication direction or a shared wire for both directions. It differs from parallel communications, which use multiple communication wires and can transfer several bits at the same time. Compared with parallel communications, serial communications provide lower speed, but allow longer cable length and are less expensive.

22.1 Universal Asynchronous Receiver and Transmitter

One of the most common usages of universal asynchronous receiver and transmitter (UART) is for exchanging data between a microprocessor and a PC serial port to debug software or monitor systems. UART also has been widely used for various peripherals, such as printers, terminals, and modems. The keyword "universal" means the serial interface is programmable. UART is often configured to communicate synchronously, which is then called USART.

The asynchronous transmission allows bits to be transmitted in a serial fashion without requiring the sender to provide a clock signal to the receiver. However, both senders and receivers must agree on the data transmission rate before the communication starts. The sender and the receiver should use the same baud rate to set up the clock agreement. In digital systems, the baud rate is the bit rate, *i.e.*, the number of bits transmitted per second. Usually, the UART interface can tolerate a clock shift up to 10% during the transmission. In some analog systems, such as modems, the baud rate is larger than the corresponding bit rate when there are more than two voltage levels and a voltage signal transmitted can represent multiple bits.

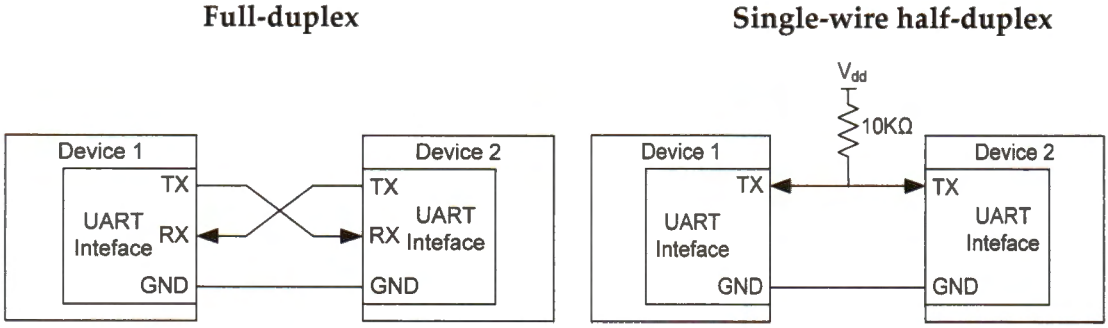


Figure 22-1. Connection between two UART devices in asynchronous mode

The transmission involves two communication lines (TX and RX), as shown in Figure 22-1.

- With full-duplex communication, data is always transmitted out bit by bit from the TX line and is received by the other device on its RX line. The receiver reassembles bits received into bytes.
- With the single-wire half-duplex communication, TX and RX are internally connected, and only one wire is used. TX is used for both sending and receiving data. In this mode, TX pins are pulled up externally because these two pins must be configured as open-drain.

For synchronous serial communication, the clock (CLK) pin of the devices must be connected. Also, the CTS (clear to send) line must connect with the RTS (request to send) line of the other device.

22.1.1 Communication Frame

UART divides data to be transmitted into frames. A frame is the smallest unit of communication. In a frame, the data length (7, 8, or 9 bits), the parity bit (even, odd, or no parity), the number of stop bits (0.5, 1, 1.5, or 2 bits), and the data order (MSB or LSB first) are configurable. Figure 22-2 shows one commonly used data frame: 8-P-1 (8 data bits, parity, 1 stop bit).

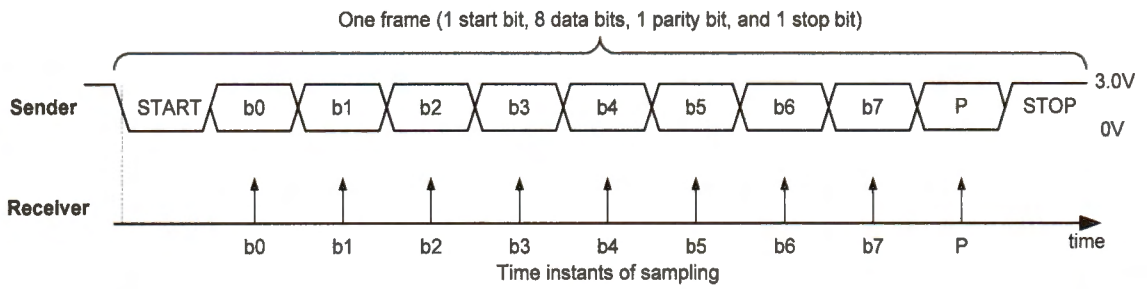


Figure 22-2. 8-P-1 frame (a start bit, eight data bits, one parity bit, and one stop bit). The least significant (LSB) of the data is sent out first in this example.

Each frame begins with a start bit, represented by a low-level voltage. After the start bit, the individual bits of each frame are shifted out of one UART interface and into another. Software can configure the data transmission order. Either the least significant bit (LSB) or the most significant bit (MSB) can be sent first. For example, suppose the LSB is sent first. When UART sends `0xE1`, the bit stream `10001111` (read from left to right) is seen on the transmission line. The number of bits in the data can be programmed to be 7, 8 or 9.

When the sender sends a frame, the sender can optionally calculate the parity of this frame and send the parity bit to the receiver for error checking. The optional parity bit helps improve the data integrity. The parity bit uses a high-level voltage to represent a logic `0` and a low-level voltage to represent a logic `1`. Software can configure logic `1` on the parity bit to represent either an odd or even number of ones in the transmitted data.

- ***Even parity***. The combination of data bits and the parity bit contains an even number of 1s.
- ***Odd parity***. The total number of 1s in the data bits and the parity bit is an odd number.

For example, if the data bits are `00010001` in binary, the parity bit will be 1 if odd parity is used, and 0 if even parity is used.

Each frame ends with a stop bit, represented by a high voltage. If no further data is transmitted, the voltage of the transmission line remains high. If the receiver does not obtain the stop bit, the current frame is considered corrupted and discarded. Additionally, the number of stop bit in each frame is usually one by default. However, it can be programmed to have 0.5, 1, 1.5, or 2 stop bits.

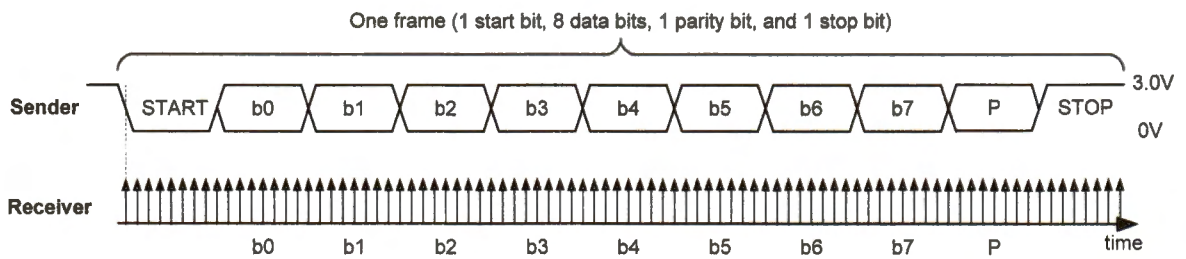


Figure 22-3. Receiver oversamples each bit 8 times

As the transmitter and receiver clocks are independent of each other, oversampling is an effective approach to mitigate the effects of clock deviation and avoid corruption by high-frequency noise. The most commonly used sampling rate is 8 or 16 times the baud rate (introduced in the next section). The receiver samples each bit 8 or 16 times and uses these values to estimate the middle of each bit pulse, resulting in a more reliable and robust transmission link.

22.1.2 Baud Rate

Historically the baud rate was used in telecommunications to represent the number of pulses or transitions physically transferred per second.

Baud Rate ≠ Bit Rate

- By using phase shift and other technologies, a pulse on phone lines can represent multiple binary bits, resulting in a bit rate larger than the baud rate.
- In digital communication systems, because each pulse represents a single bit, the baud rate is the number of bits physically transferred per second, including the actual data content and the protocol overhead, leading to a bit rate lower than the baud rate.

For example, if the baud rate is 9600, and an 8-N-1 frame consists of a start bit, 8 data bits, a stop bit, and no parity bit, then the transmission rate of actual data is not 9600 bits per second/8 = 1200 bytes per second. Instead, it is 9600/(1 + 8 + 1) = 960 bytes per second. The start and stop bits are the protocol overhead.

On STM32L4, the baud rate is calculated as follows:

$$Baud\ Rate = \frac{(1 + OVER8) \times f_{PCLK}}{USARTDIV}$$

where f_{PCLK} is the clock frequency of the processor. The divider *USARTDIV* is stored in the Baud Rate Register (`BRR`) . The value of `OVER8` is defined as follows.

$$OVER8 = \begin{cases} 0, & Signal\ is\ oversampled\ by\ 16 \\ 1, & Signal\ is\ oversampled\ by\ 8 \end{cases}$$

Also, the divider *USARTDIV* can be calculated from `BRR`.

$$USARTDIV = \begin{cases} BRR, & Signal\ is\ oversampled\ by\ 16 \\ BRR[15:4] \times 16 + BRR[2:0] \times 2, & Signal\ is\ oversampled\ by\ 8 \end{cases}$$

Example 1: Oversampling by 16, processor core 80 MHz, baud rate = 9600. Find *BRR*.

$$OVER8 = 0$$

$$USARTDIV = \frac{(1 + OVER8) \times f_{PCLK}}{Baud\ Rate} = \frac{80000000}{9600} = 8333.33 \approx 8333$$

$$BRR = USARTDIV = 8333 = 0x208D$$

Example 2: Oversampling by 8, processor core 80 MHz, baud rate = 9600. Find *BRR*.

$$OVER8 = 1$$

$$USARTDIV = \frac{(1 + OVER8) \times f_{PCLK}}{Baud\ Rate} = \frac{2 \times 80000000}{9600} = 16666.67$$
$$\approx 16667$$

The hex equivalent of 1667 is `0x411B`.

$$BRR[3:0] = USARTDIV[3:0] \gg 1 = 0xB \gg 1 = 0x5$$

$$BRR[15:4] = USARTDIV[15:4] = 0x411$$

$$BRR = BRR[15:4]:BRR[3:0] = 0x4115$$

22.1.3 UART Standards

Voltage signals for UART are defined in different standards, such as RS-232, RS-422, and RS-485. The prefix RS stands for "recommended standard." Table 22-1 compares these three standards. While the voltage of the TX and RX line in RS-422 and RS-485 is differential, with two separate wires for each line, RS-232 uses a single-ended voltage with a shared ground.

Figure 22-4 compares ***single-ended*** and ***differential signaling***. Besides the shared ground, single-ended signaling uses just one wire to transmit signals. Differential signaling uses two twisted wires with equal but opposite signals to transmit digital data. Electrical noise can be inducted into the signal wires or can be generated by the voltage difference between two ground references. Noise is coupled into both wires equally. Therefore, the noise can be canceled out at the receiver. Compared with single-ended signaling, differential signaling can transmit a higher frequency signal over a greater distance.

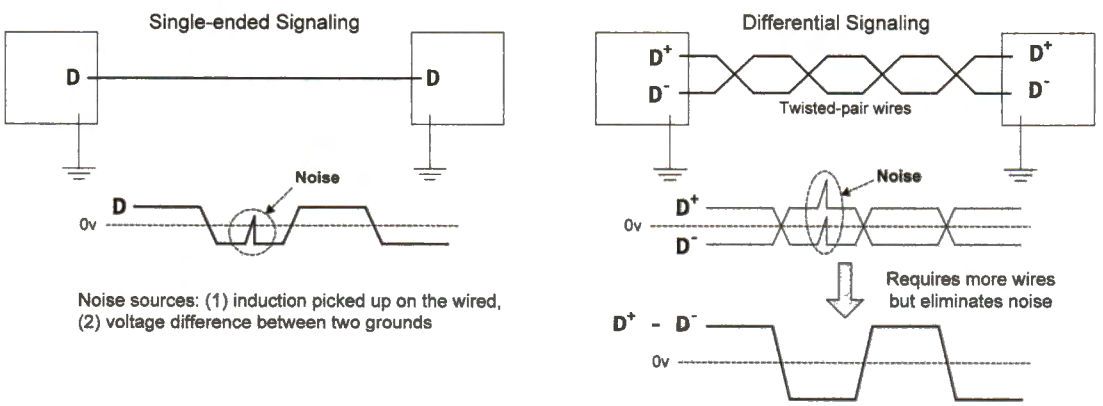


Figure 22-4. Comparison of single-ended and differential signaling

In RS-232, a voltage signal between +5V and +15V represents a logic one being transmitted, whereas a signal between -5V and -15V represents a logic zero. The receiver must interpret a voltage with +3V and +25V as a logic one, and a voltage with -3V to -25V as a logic zero. Any voltage signals between -3V and + 3V are invalid data. When the line is idle, the line must be driven to logic zero.

	RS-232	RS-422	RS-485
Voltage signal	Single-ended (logic 1: +5 to +15V, logic 0: -5 to -15 V)	Differential (-6V to +6V)	Differential (-7V to +12V)
Max distance	50 feet	4000 feet	4000 feet
Max speed	20 Kbit/s	10 Mbit/s	10 Mbit/s
Number of devices	1 master, 1 receiver	1 master, 10 receivers	32 masters, 32 receivers
Mode	Full duplex	Full duplex, half duplex	Full duplex, half duplex

Table 22-1. Comparing popular UART interfaces

Most modern computers only provide USB ports, not UART ports. Some old computers have RS-232 serial ports. However, we cannot directly connect an STM32 processor to the RS-232 port on a computer due to the voltage incompatibility. The STM32 processors can only tolerate voltage signals under 5V. Additionally, STM32 uses 0V to represent logic zero and 3V to represent logic one.

The FT232R chip converts a UART port to a standard USB interface, as shown below.

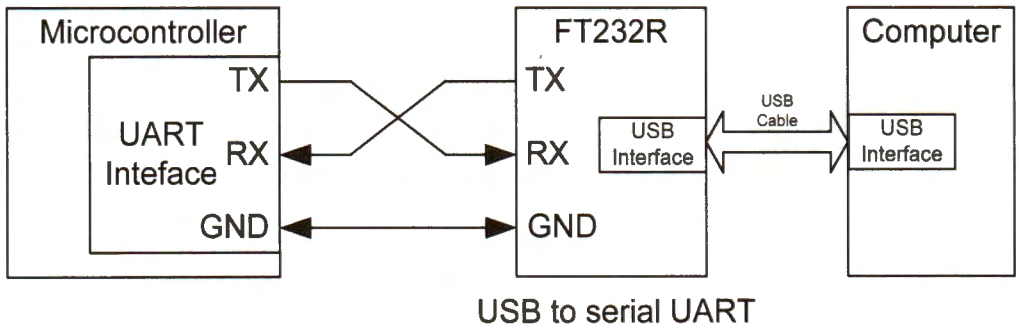


Figure 22-5. Serial communication via a USB-to-UART converter

The following diagram displays the voltage signal of the UART port when transmitting two data bytes, `0x32`, and `0x3C`. Each data frame includes one start bit, 8-bit data, and one stop bit. No parity bit is used in this example. After the start bit, the least significant bit

of the data is transmitted first. For example, the binary value of `0x32` is `0b00110010`, and the bit sequence seen on the transmission (TX) line is `01001100`. The baud rate is set to 9,600, and thus each bit takes approximately 0.104ms. When the TX line is idle, the voltage on it is 3V. The start bit has 0V while the stop bit has 3V.

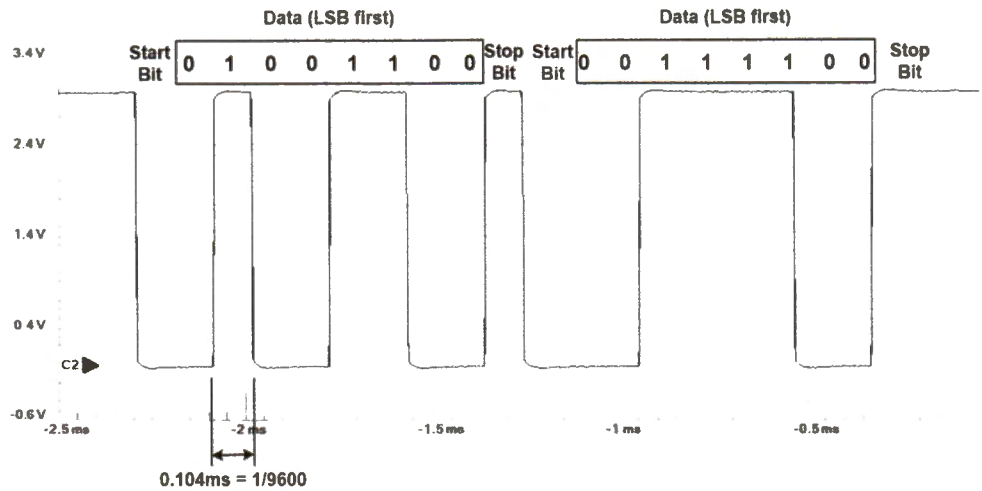


Figure 22-6. Voltage signal when transmitting 0x32 and 0x3C via UART (1 start bit, 1 stop bit, 8 data bits, no parity, baud rate = 9,600)

22.1.4 UART Communication via Polling

The following sections introduce how to send or receive data via UART ports by using three different methods: polling, interrupt, and DMA. Polling is the simplest but most inefficient method. The interrupt approach is more efficient but not suitable for high data transfer rates. The DMA method is complex but the most effective.

Figure 22-7 shows the connection of two UART ports between two processors. The TX pin of one processor is connected to the RX pin of the other processor, and vice versa. Because the polling method blocks the processor from running other tasks, software cannot use polling to send and receive data simultaneously.

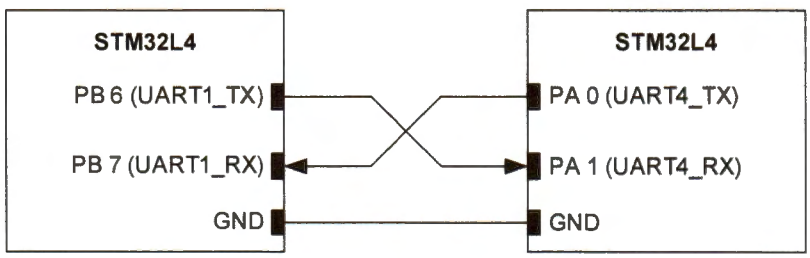


Figure 22-7. Connecting two UART ports

The code in Example 22-1 initializes a UART port in asynchronous mode (no hardware flow control) with oversampling by 16. Assume the UART clock is 80 MHz, and the baud rate is 9600. The data frame consists of 8 data bits, 1 start bit, 1 stop bit, and no parity bit. Software can initialize the UART ports using the following functions.

```
USART_Init(UART4);
USART_Init(USART1);
```

UART4 and USART1 are `struct` variables defined in the device header file (`stm32l476xx.h`).

```
void USART_Init (USART_TypeDef * USARTx) {

  // Disable USART
  USARTx->CR1 &= ~USART_CR1_UE;

  // Set data length to 8 bits
  // 00 = 8 data bits, 01 = 9 data bits, 10 = 7 data bits
  USARTx->CR1 &= ~USART_CR1_M;

  // Select 1 stop bit
  // 00 = 1 Stop bit      01 = 0.5 Stop bit
  // 10 = 2 Stop bits     11 = 1.5 Stop bit
  USARTx->CR2 &= ~USART_CR2_STOP;

  // Set parity control as no parity
  // 0 = no parity,
  // 1 = parity enabled (then, program PS bit to select Even or Odd parity)
  USARTx->CR1 &= ~USART_CR1_PCE;

  // Oversampling by 16
  // 0 = oversampling by 16, 1 = oversampling by 8
  USARTx->CR1 &= ~USART_CR1_OVER8;

  // Set Baud rate to 9600 using APB frequency (80 MHz)
  // See Example 1 in Section 22.1.2
  USARTx->BRR = 0x208D;

  // Enable transmission and reception
  USARTx->CR1 |= (USART_CR1_TE | USART_CR1_RE);

  // Enable USART
  USARTx->CR1 |= USART_CR1_UE;

  // Verify that USART is ready for transmission
  // TEACK: Transmit enable acknowledge flag. Hardware sets or resets it.
  while ((USARTx->ISR & USART_ISR_TEACK) == 0);

  // Verify that USART is ready for reception
  // REACK: Receive enable acknowledge flag. Hardware sets or resets it.
  while ((USARTx->ISR & USART_ISR_REACK) == 0);
}
```

Example 22-1. Initializing a UART port

Example 22-2 selects the system clock to drive USART1 and UART4.

```
int main(void){

  // Enable GPIO clock and configure the Tx pin and the Rx pin as:
  // Alternate function, High Speed, Push-pull, Pull-up

  //------------------  GPIO Initialization for USART 1 -----------------
  // PB.6 = AF7 (USART1_TX), PB.7 = AF7 (USART1_RX), See Appendix I
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // Enable GPIO port B clock

  // 00 = Input, 01 = Output, 10 = Alternate Function, 11 = Analog
  GPIOB->MODER   &= ~(0xF << (2*6)); // Clear mode bits for pin 6 and 7
  GPIOB->MODER   |=   0xA << (2*6);  // Select Alternate Function mode

  // Alternative function 7 = USART 1
  // Appendix I shows all alternate functions
  GPIOB->AFR[0]  |=   0x77 << (4*6); // Set pin 6 and 7 to AF 7

  // GPIO Speed: 00 = Low speed,  01 = Medium speed,
  //             10 = Fast speed, 11 = High speed
  GPIOB->OSPEEDR |=   0xF<<(2*6);

  // GPIO Push-Pull: 00 = No pull-up/pull-down, 01 = Pull-up (01)
  //                 10 = Pull-down, 11 = Reserved
  GPIOB->PUPDR   &= ~(0xF<<(2*6));
  GPIOB->PUPDR   |=   0x5<<(2*6); // Select pull-up

  // GPIO Output Type: 0 = push-pull, 1 = open drain
  GPIOB->OTYPER  &=  ~(0x3<<6);

  //------------------  GPIO Initialization for USART 4 -----------------
  // PA.0 = AF8 (UART4_TX), PA.1 = AF8 (UART4_RX), See Appendix I
  // The code is very similar to the one given above.
  ...

  RCC->APB2ENR  |= RCC_APB2ENR_USART1EN;   // Enable UART 1 clock
  RCC->APB1ENR1 |= RCC_APB1ENR1_UART4EN;   // Enable UART 4 clock

  // Select system clock (SYSCLK) USART clock source of UART 1 and 4
  // 00 = PCLK,   01 = System clock (SYSCLK),
  // 10 = HSI16,  11 = LSE
  RCC->CCIPR &= ~ (RCC_CCIPR_USART1SEL | RCC_CCIPR_UART4SEL);
  RCC->CCIPR |= (RCC_CCIPR_USART1SEL_0 | RCC_CCIPR_UART4SEL_0);

  USART_Init(USART1);
  USART_Init(UART4);
  ...
}
```

Example 22-2. Enable and select the clock of UART ports

When UART receives a byte, hardware sets the receive register not empty flag (RXNE) in the status register (ISR). In the polling approach, software constantly checks the RXNE flag and reads the receive data register (RDR) once it is set. Reading register RDR clears the RXNE flag automatically.

Example 22-3 shows the implementation of receiving data by polling. This polling method is inefficient, and the `while` loop prevents the processor from running other tasks.

```
void USART_Read (USART_TypeDef *USARTx, uint8_t *buffer, uint32_t nBytes) {

  int i;

  for (i = 0; i < nBytes; i++) {
    while (!(USARTx->ISR & USART_ISR_RXNE)); // Wait until hardware sets RXNE
    buffer[i] = USARTx->RDR;                 // Reading RDR clears RXNE
  }
}
```

Example 22-3. Receive data from a UART port via busy polling

When UART sends a byte, software must wait until the TxE (transmission data register empty) flag is set in the status register (ISR). Hardware sets the TxE flag when the content of the transmission data register (TDR) has been transferred into the shift register. Additionally, writing to the USART data register (DR) clears the TxE flag automatically. After exiting the *for* loop, software must wait for the transmission complete (TC) flag to ensure the last byte has been sent out.

Example 22-4 shows the implementation of sending data by polling. Again, the `while` loop prevents the processor from performing other tasks.

```
void USART_Write (USART_TypeDef *USARTx, uint8_t *buffer, uint32_t nBytes) {

  int i;

  for (i = 0; i < nBytes; i++) {
    while (!(USARTx->ISR & USART_ISR_TXE)); // Wait until hardware sets TXE
    USARTx->TDR = buffer[i] & 0xFF;         // Writing to TDR clears TXE flag
  }

  // Wait until TC bit is set. TC is set by hardware and cleared by software.
  while (!(USARTx->ISR & USART_ISR_TC));    // TC: Transmission complete flag

  // Writing 1 to the TCCF bit in ICR clears the TC bit in ISR
  USARTx->ICR |= USART_ICR_TCCF; // TCCF: Transmission complete clear flag
}
```

Example 22-4. Send data out via a UART port via busy polling

22.1.5 UART Communication via Interrupt

An USART interrupt can be generated upon the occurrence of several events, such as transmission data register empty (TxE), transmission complete (TC), received data register not empty (RXNE), overrun error detected (ORE), idle line detected (IDLE), and parity error (PE).

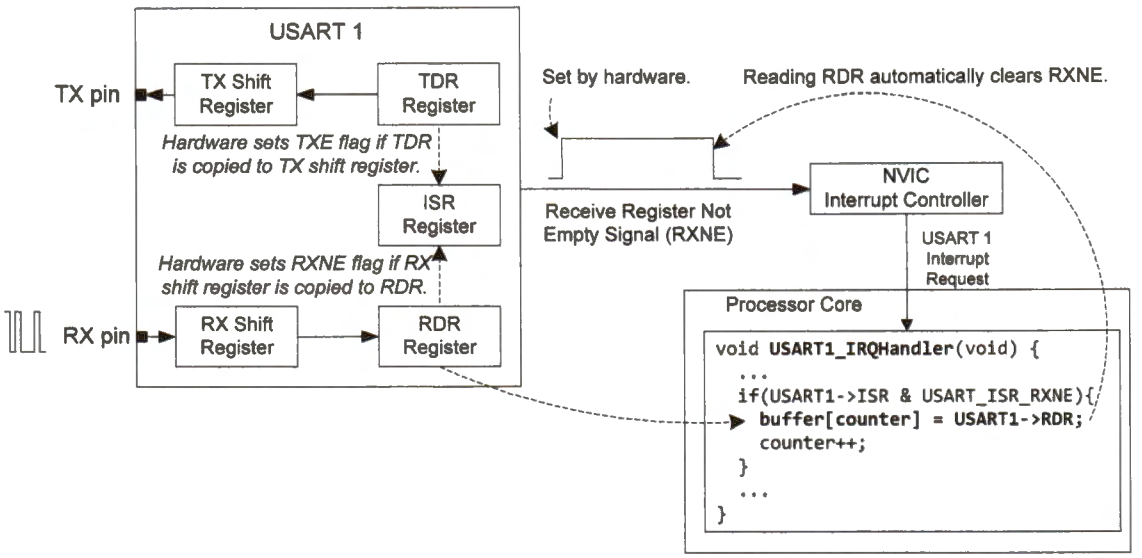


Figure 22-8. Using interrupt to receive data from USART 1

When UART receives a byte, an interrupt request is generated, and the processor responds to the request by executing the corresponding UART interrupt handler. The interrupt handler reads the receive data register (RDR) and copies it to the next empty buffer, as shown in Figure 22-8. Because several UART events can generate interrupts, the interrupt handler must check whether an RXNE event has taken place.

Software must enable UART interrupts to send or receive data, as shown in Example 22-5.

```
USART1->CR1 |= USART_CR1_RXNEIE;    // Receive register not empty interrupt
USART1->CR1 &= ~USART_CR1_TXEIE;    // Transmit register empty interrupt
NVIC_SetPriority(USART1_IRQn, 0);   // Set the highest urgency
NVIC_EnableIRQ(USART1_IRQn);        // Enable NVIC interrupt
```

Example 22-5. Enable UART sending and receiving interrupts

Example 22-6 shows the implementation of receiving data from UART by using interrupts. There are two global counter variables to record the number of bytes that have been received. The *receive()* function is generic, and is called by different UART interrupt handlers. Therefore, it takes three input arguments to differentiate UART ports, the receive buffers, and the byte counters.

```
#define BufferSize 32
uint8_t USART1_Buffer_Rx[BufferSize], USART4_Buffer_Rx[BufferSize];
volatile uint32_t Rx1_Counter = 0, Rx4_Counter = 0;

void USART1_IRQHandler(void) {
  receive(USART1, USART1_Buffer_Rx, &Rx1_Counter);
}

void UART4_IRQHandler(void) {
  receive(UART4, USART4_Buffer_Rx, &Rx4_Counter);
}

void receive(USART_TypeDef *USARTx, uint8_t *buffer, uint32_t *pCounter) {
  if(USARTx->ISR & USART_ISR_RXNE) {   // Check RXNE event
     buffer[*pCounter] = USARTx->RDR;  // Reading RDR clears the RXNE flag
     (*pCounter)++;                    // Dereference and update memory value
     if((*pCounter) >= BufferSize) {   // Check buffer overflow
        (*pCounter) = 0;               // Circular buffer
     }
  }
}
```

Example 22-6. Receiving data from a UART port via interrupt

Example 22-7 gives a generic implementation to transmit data via a UART port by using interrupts. For example, software can send out the data by executing the following statement: `UART_Send(USART1,buffer)`, which writes only the first byte to the transmit data register (`TDR`). This will start the transmission process. This function will immediately return to the caller after enabling `TXE` interrupt and write the first byte to the transmit data register (`TDR`). This allows the caller to continue to execute other tasks while the transmission is being performed in the background, as shown in Figure 22-9.

An interrupt will be generated after each byte has been sent. The interrupt handler writes the next byte to `TDR` to start the next transmission. This process repeats until a total of `BufferSize` bytes have been sent. The interrupt disables the interrupt for the `TXE` events.

```
volatile uint32_t Tx1_Counter = 0, Tx4_Counter = 0;

void UART_Send (USART_TypeDef *USARTx, uint8_t *buffer){
   USARTx->CR1 |= USART_CR1_TXEIE; // Enable TXE Interrupt
   // Write to Transmit Data Register (TDR) to start transmission
   // An interrupt will be initiated after data in TDR has been sent.
   USARTx->TDR = buffer[0];
}

void USART1_IRQHandler(void) {
  send(USART1, USART1_Buffer_Tx, &Tx1_Counter);
}
```

```
void UART4_IRQHandler(void) {
  send(UART4, USART4_Buffer_Rx, &Tx4_Counter);
}

void send(USART_TypeDef *USARTx, uint8_t *buffer, uint32_t *pCounter){
  if(USARTx->ISR & USART_ISR_TXE) {            // Check TXE flag
     (*pCounter)++;                            // Bytes that have been sent
     if(*pCounter <= BufferSize - 1) {         // Transmit the next byte
        USARTx->TDR = buffer[pCounter] & 0xFF; // Writing to TDR clears TXE
     } else {                                  // Transmission completes
        (*pCounter) = 0;                       // Clear the counter
        USARTx->CR1 &= ~USART_CR1_TXEIE;       // Disable TXE interrupt
     }
  }
}
```

Example 22-7. Send data out via a UART port by using interrupt

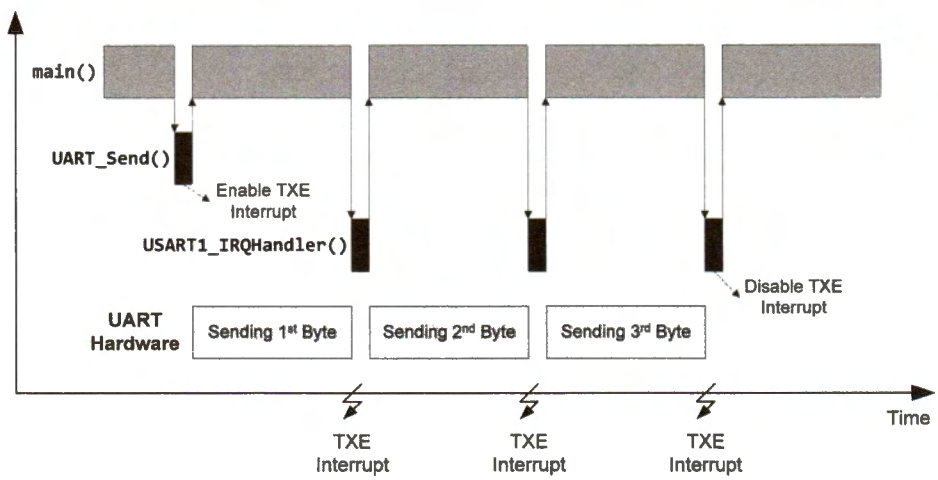


Figure 22-9. Sending three bytes via USART1 by using interrupts

Due to the non-blocking feature of interrupt, we can send and receive data simultaneously between two UART ports on the same processor, as shown in Figure 22-10.

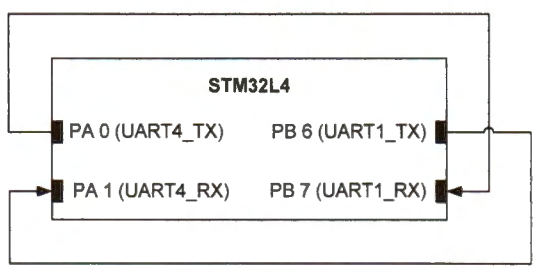


Figure 22-10. Communication between two ports on the same processor

22.1.6 UART Communication via DMA

Using a direct memory access (DMA) controller to move data between a buffer and UART data registers is the most efficient way to perform UART communication. Chapter 19 introduces DMA in detail.

Figure 22-11 shows the basic idea. As shown in Table 19-2, `USART1_TX` and `USART1_RX` can be connected to channels 6 and 7 of DMA controller 2, respectively. A `TXE` or `RXNE` event triggers a DMA request on channel 6 and channel 7, respectively.

- Whenever the `TXE` bit is set in the `ISR` register, DMA controller 2 transfers one byte via channel 6 from the memory buffer (pointed to by the `CMAR` register of DMA 2 channel 6) to the *transmit data register* `TDR` (pointed to by the `CPAR` register of DMA 2 channel 6).
- Similarly, whenever the `RXNE` bit is set in the `ISR` register, DMA controller 2 transfers one byte via channel 7 from the *receive data register* `RDR` to the buffer.

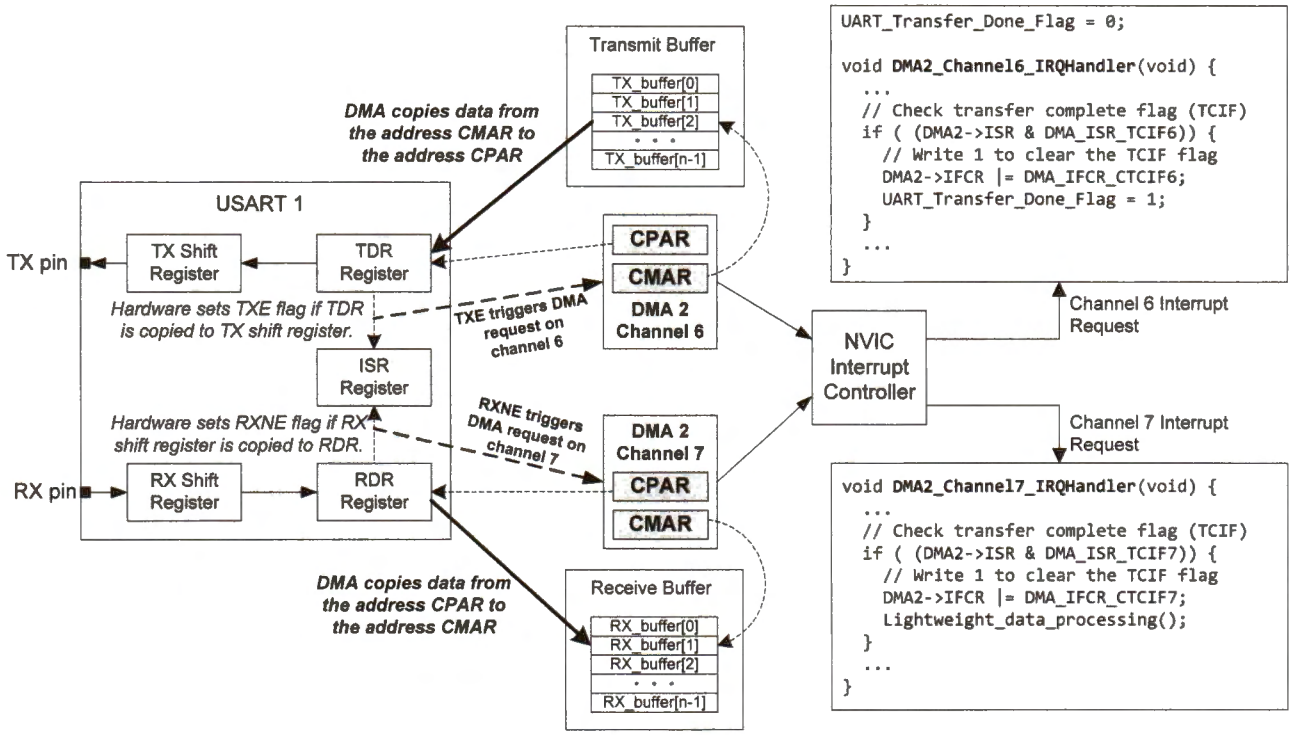


Figure 22-11. Using DMA to receive data from USART 1

To enable DMA for the transmission, software must set the `DMAT` bit in the `CR3` register.

```
USART1->CR3 |= USART_CR3_DMAT;
```

To enable DMA for the reception, software must set the `DMAR` bit in the `CR3` register.

```
USART1->CR3 |= USART_CR3_DMAR;
```

If DMA interrupts are enabled, the DMA controller can generate interrupt requests to execute the corresponding interrupt handler.

Example 22-8 and Example 22-9 give C code that configures channel 6 and 7 of DMA controller 2 to serve TX and RX of UART 1, respectively. Data transmission or reception takes place immediately when *UART1_DMA_Transmit* or *UART1_DMA_Receive* is called.

```
void UART1_DMA_Transmit (uint8_t *pBuffer, uint32_t size) {
  RCC->AHB1ENR |= RCC_AHB1ENR_DMA2EN;      // Enable DMA2 clock
  DMA2_Channel6->CCR &= ~DMA_CCR_EN;       // Disable DMA channel
  DMA2_Channel6->CCR &= ~DMA_CCR_MEM2MEM; // Disable memory to memory mode
  DMA2_Channel6->CCR &= ~DMA_CCR_PL;       // Channel priority level
  DMA2_Channel6->CCR |=  DMA_CCR_PL_1;     // Set DMA priority to high
  DMA2_Channel6->CCR &= ~DMA_CCR_PSIZE;    // Peripheral data size 00 = 8 bits
  DMA2_Channel6->CCR &= ~DMA_CCR_MSIZE;    // Memory data size: 00 = 8 bits
  DMA2_Channel6->CCR &= ~DMA_CCR_PINC;     // Disable peripheral increment mode
  DMA2_Channel6->CCR |=  DMA_CCR_MINC;     // Enable memory increment mode
  DMA2_Channel6->CCR &= ~DMA_CCR_CIRC;     // Disable circular mode
  DMA2_Channel6->CCR |=  DMA_CCR_DIR;      // Transfer direction: to peripheral
  DMA2_Channel6->CCR |=  DMA_CCR_TCIE;     // Transfer complete interrupt enable
  DMA2_Channel6->CCR &= ~DMA_CCR_HTIE;     // Disable Half transfer interrupt
  DMA2_Channel6->CNDTR = size;             // Number of data to transfer
  DMA2_Channel6->CPAR  = (uint32_t)&(USART1->TDR);  // Peripheral address
  DMA2_Channel6->CMAR  = (uint32_t) pBuffer;        // Transmit buffer address
  DMA2_CSELR->CSELR &= ~DMA_CSELR_C6S;     // See Table 19-2
  DMA2_CSELR->CSELR |= 2<<20;              // Map channel 6 to USART1_TX
  DMA2_Channel6->CCR |= DMA_CCR_EN;        // Enable DMA channel
}
```

Example 22-8. Configure DMA 2 channel 6 for UART 1 transmit

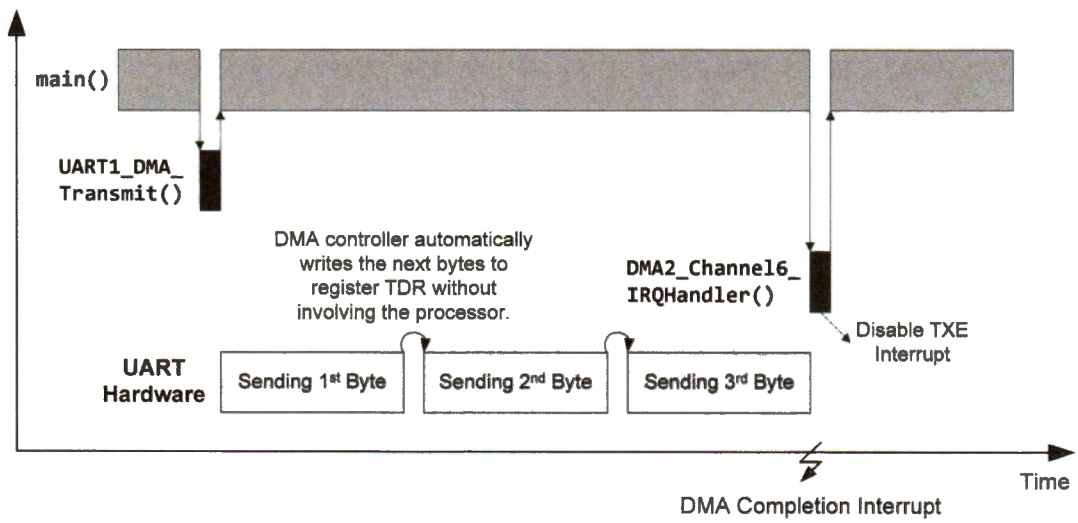


Figure 22-12. Sending three bytes via DMA.

```
void UART1_DMA_Receive (uint8_t *pBuffer, uint32_t size) {
  RCC->AHB1ENR |= RCC_AHB1ENR_DMA2EN;     // Enable DMA2 clock
  DMA2_Channel7->CCR &= ~DMA_CCR_EN;      // Disable DMA channel
  DMA2_Channel7->CCR &= ~DMA_CCR_MEM2MEM;// Disable memory to memory mode
  DMA2_Channel7->CCR &= ~DMA_CCR_PL;      // Channel priority level
  DMA2_Channel7->CCR |=  DMA_CCR_PL_1;    // Set DMA priority to high
  DMA2_Channel7->CCR &= ~DMA_CCR_PSIZE;   // Peripheral data size 00 = 8 bits
  DMA2_Channel7->CCR &= ~DMA_CCR_MSIZE;   // Memory data size: 00 = 8 bits
  DMA2_Channel7->CCR &= ~DMA_CCR_PINC;    // Disable peripheral increment mode
  DMA2_Channel7->CCR |=  DMA_CCR_MINC;    // Enable memory increment mode
  DMA2_Channel7->CCR &= ~DMA_CCR_CIRC;    // Disable circular mode
  DMA2_Channel7->CCR &= ~DMA_CCR_DIR;     // Transfer direction: to memory
  DMA2_Channel7->CCR |=  DMA_CCR_TCIE;    // Transfer complete interrupt enable
  DMA2_Channel7->CCR &= ~DMA_CCR_HTIE;    // Disable Half transfer interrupt
  DMA2_Channel7->CNDTR = size;            // Number of data to transfer
  DMA2_Channel7->CPAR  = (uint32_t)&(USART1->RDR); // Peripheral address
  DMA2_Channel7->CMAR  = (uint32_t) pBuffer;       // Receive buffer address
  DMA2_CSELR->CSELR &= ~DMA_CSELR_C6S;    // See Table 19-2
  DMA2_CSELR->CSELR |= 2<<24;             // Map channel 7 to USART1_RX
  DMA2_Channel7->CCR |= DMA_CCR_EN;       // Enable DMA channel
}
```

Example 22-9. Configure DMA 2 channel 7 for UART 1 receive

Comparing Figure 22-9 and Figure 22-12, we can see that DMA is more efficient than the interrupt approach. When DMA completes, a DMA interrupt request will be generated. The DMA interrupt handler can change the completion flag, as shown below.

```
volatile uint8_t TransmissionCompleteFlag = 0;

void DMA2_Channel7_IRQHandler(void) { // USART1_RX

  if ( (DMA2->ISR & DMA_ISR_TCIF7) == DMA_ISR_TCIF7 ) {
    // Write 1 to clear the corresponding TCIF flag
    DMA2->IFCR |= DMA_IFCR_CTCIF7;
    TransmissionCompleteFlag = 1;
  }

  if ( (DMA2->ISR & DMA_ISR_HTIF7) == DMA_ISR_HTIF7 ) // half transfer
    DMA2->IFCR |= DMA_IFCR_CHTIF7;

  if ( (DMA2->ISR & DMA_ISR_GIF7) == DMA_ISR_GIF7 )   // global interrupt
    DMA2->IFCR |= DMA_IFCR_CGIF7;

  if ( (DMA2->ISR & DMA_ISR_TEIF7) == DMA_ISR_TEIF7 ) // transfer error
    DMA2->IFCR |= DMA_IFCR_CTEIF7;
}
```

Example 22-10. DMA interrupt handler

22.1.7 Serial Communication to Bluetooth Module

Bluetooth is a low-power, low-cost wireless communication protocol operating in the 2.4GHz band, which is a globally unlicensed radio frequency band for industry, science and medical (ISM) applications.

Bluetooth is based on a master-slave model. Up to seven active slave devices can be connected to the master to form a local network (called a ***piconet***), while there may be other inactive slave devices in the piconet. The disadvantages of Bluetooth include that it has a short communication range (15-30 feet) and it is relatively insecure.

Bluetooth uses a technique of time division duplex (TDD) to coordinate the access to the physical channel. It divides the time into a fixed length interval (625 microseconds) called ***slots***. Each second is divided into 1,600 slots. The master sets the ratio frequency and initiates a data communication.

- *Master sets the frequency hopping sequence.* Bluetooth divides the 2.4GHz band into 79 channels with their frequencies 1 MHz apart. To avoid interference on one specific channel, in each slot both the master and the slave device switch to a different channel. Such frequency hopping is performed 1,600 times per second, which allows a retransmission to occur soon on a different channel, hopefully on a clean one without any interference. We call this technique FHSS (frequency hopping spread spectrum). The master sets the frequency hopping sequence, and the slave devices follow the hopping sequence by using a special algorithm.
- *Master initiates communication.* The master transmits data to a slave in even numbered slots and receives data from a slave in odd numbered slots. A slave device is only allowed to transmit data in a slot if the master has polled it or sent a data packet to it in the preceding slot. The slave then responds to the master by sending a data packet or a `NULL` packet if the slave has nothing to send.
- Each Bluetooth device has a unique fixed 48-bit address, which is assigned at manufacture time. Two communicating devices need to know each other's address. Software programs can specify the address if programmers know it. The address can also be discovered during the device discovery process. The discovery process searches all devices nearby and identifies the one that matches a user-friendly name such as "Joe's Phone."

22.1.7.1 Bluetooth Transfer Protocols

There are many transfer protocols available between two Bluetooth devices. A standardized set of protocols designed for a particular type of device is termed as a Bluetooth device profile. The following gives a few examples.

- The radio frequency communication (RFCOMM) profile emulates the serial cable line and guarantees reliable delivery of every packet. It is used for devices such as PC, printers, and modems.
- The generic audio/video distribution profile (GAVDP) is used to stream video or audio data to devices such as stereo headphones and laptops.
- The audio/video remote control profile (AVRCP) provides a standard interface to control audio or video devices. Example devices include headphones, speakers, and TVs.

In this section, we focus on the RFCOMM profile designed for serial communication. There are inexpensive Bluetooth-UART modules such as the HC-05 (master or slave) and HC-06 (slave only) that support the RFCOMM profile. They include a UART interface for serial communication between the Bluetooth module and microprocessors, and a Bluetooth interface for wireless communication between two Bluetooth modules, as shown in Figure 22-13.

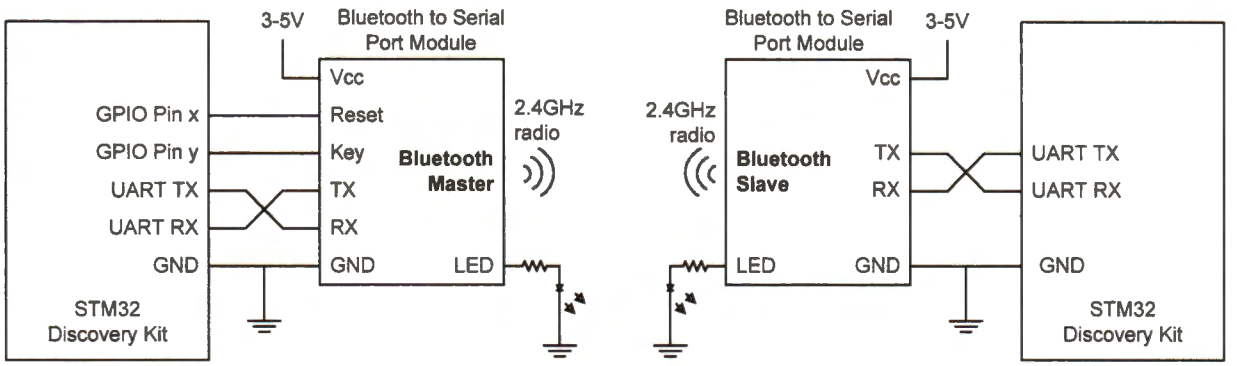


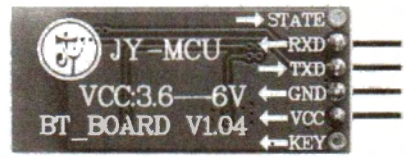
Figure 22-13. Serial port Bluetooth modules. Bluetooth modules communicate with each other in wireless. Each Bluetooth module transfers data to or from the processor via UART.

22.1.7.2 Pairing Bluetooth Modules

Two Bluetooth modules need to be paired and bound together before they can exchange data. The master device must know the 48-bit address of the slave device and the pairing password set by the slave device.

The microprocessor needs to send string commands via the serial interface to the Bluetooth master device to complete the pairing and binding process. These string commands start with "`AT`" (abbreviation for "attention") and end with a terminator "return", *i.e.*, with ASCII values of "`0x0D,0x0A`". Therefore, they also called "AT commands".

The following shows the procedure for configuring and pairing Bluetooth modules. It also lists frequently used AT commands that the microprocessor performs to set up the Bluetooth master module. In this example, we assume that the slave Bluetooth module has a MAC address of "0018,e4,0c680a" and a pairing password of "1234".



1. Pull the KEY pin high to put the Bluetooth master device in command mode

2. Pull the RESET pin low for a short time and then pull it high to reset the Bluetooth master module

3. Software sends the following commands via the UART interface to the Bluetooth master module. (The quote signs are not part of the command.)

```
"AT+RESET\r\n"              ; add a return, i.e., "0x0D,0x0A", to the end
                            ; ASCII 0x0D = CR (carriage return)
                            ; ASCII 0x0A = NL (new line),

"AT+UART=57600,0,0\r\n"     ; Baud rate: 57,600 bits/s, 1 stop bit, no parity bit
                            ; 1st parameter: baud rate
                            ; 2nd parameter: 0 = 1 stop bit
                            ;                1 = 2 stop bits
                            ; 3rd parameter: 0 = no parity bit
                            ;                1 = parity bit

"AT+ROLE=1\r\n"             ; A Bluetooth module can be either master or slave.
                            ; Set the module as a Bluetooth master.
                            ; 0 = Slave,
                            ; 1 = Master,
                            ; 2 = Slave Loop

"AT+PSWD=1234\r\n"          ; Password code for pairing. The password is set by
                            ; the Bluetooth slave module.

"AT+PAIR=0018,e4,0c680a\r\n" ; Paired with a 48-bit slave address

"AT+BIND=0018,e4,0c680a\r\n" ; Bound with the target slave device

"AT+CMODE=0\r\n"            ; 0 = Connect to the specified address
                            ; 1 = Connect to any address
                            ; 2 = Slave Loop
```

4. Pull the KEY pin low to let the Bluetooth master device exit command mode and enter communication mode

5. Pull the RESET pin low for a short time and then pull it high to reset the Bluetooth master module

Table 22-2. Process of using AT commands to pair two Bluetooth devices

22.2 Inter-Integrated Circuit (I²C)

Inter-integrated circuit (I²C) is a standard bus protocol, originally developed by Philips in the late 1980s. It enables the communication between microprocessors and their peripheral devices by using two wires: a ***serial data line*** (SDA) and a ***serial clock line*** (SCL). The two-wire design reduces the number of physical pins, making it inexpensive and simple to interface. The data transfer rate of I²C can be up to 100 Kbit/s in standard mode, up to 400 Kbit/s in fast mode, and up to 3.4 Mbit/s in high-speed mode.

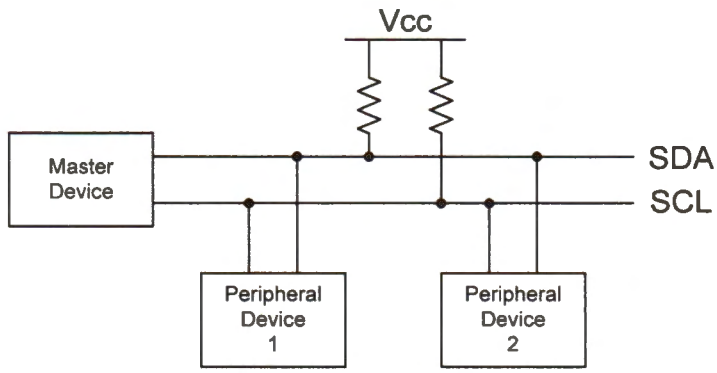


Figure 22-14. An example of I²C bus that connects one master and two peripheral devices. It uses two bidirectional open-drain wires: the serial data (SDA) line and the serial clock (SCL) line. Both wires are pulled up to a positive voltage supply. The combination of open-drain and external pull-up realizes a Boolean AND logic.

Each device, including master devices and peripheral devices, has a unique address, which typically has 7 bits, 10 bits, or 16 bits. Depending on the function, each device can serve as a transmitter, a receiver, or both. For example, a digital temperature sensor may operate only as a transmitter, an LCD driver may operate only as a receiver, and a memory device can be both a transmitter and a receiver.

The master device initializes a data transfer on the bus, and then generates the clock signals to permit that data transfer, and finally terminates the transfer after receiving all requested information. The device addressed by the master is called a ***slave***. Multiple masters can co-exist on the same I²C bus. When multiple masters want to control the bus, an arbitration procedure is then performed. The bus capacitance limits the number of devices that can be connected to the bus.

22.2.1 I²C Pins

The pins of the master and peripheral devices connected to the SDA and SCL lines should be internally configured as ***open-drain***, also known as ***open collector*** (see chapter 14.4.2).

An open-drain is a special type of output. The output pin connects to a positive voltage source if an active high (logic 1) is outputted. The output pin is in a high impedance state if a low (logic 0) is outputted. The high impedance is often achieved by keeping the output floated, *i.e.*, not connected to either the ground or the positive voltage source.

Software can configure the SDA and SCL pins of the processor as open-drained. However, the pull-up resistor within the processor is too large, often in the order of 100 kΩ. Such a large resistor provides pull-up power that is too weak for I²C. To reduce the rise time of the I²C lines, smaller resistors, such as 3 kΩ, are often used.

As shown in Figure 22-14, the SDA and SCL lines are connected to a positive supply voltage via two small pull-up resistors. The recommended resistance value is 4.7 kΩ for low speed, 3 kΩ for standard speed, and 1 kΩ for the fast speed.

22.2.2 I²C Protocol

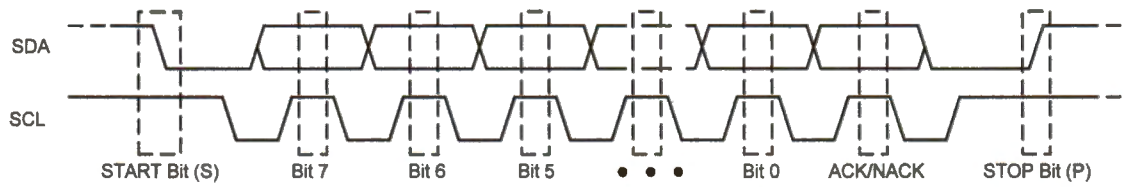


Figure 22-15. The timing diagram of sending N bits with the start bit (S) and the stop bit (P). The most significant bit (MSB) is sent out first. SDA can be updated when SCL is low. SDA must be held stable when SCL is high. The hexagonal shape on SDA line means SDA can be either high or low during that period.

The communication begins with a START bit (S) and terminates by a STOP bit (P).

- A START bit is defined as a high-to-low transition of SDA while SCL is high.
- A STOP bit is defined as a low-to-high transition of SDA while SCL is high.

The master generates both START and STOP bits. The I²C interfacing hardware of all peripheral devices is capable of detecting START and STOP. After the START bit, the master begins to send data byte by byte. For each byte, the most significant bit is transferred first. The slave sends an acknowledge bit to the master, informing the master that the slave has successfully received a byte.

After a byte is transferred, the receiver should answer the transmitter with either an acknowledge (ACK) bit or a not acknowledge (NACK) bit, as shown in Figure 22-16. The transmitter releases the SDA line during the acknowledge clock period (the ninth clock period) so that the receiver can pull SDA low. If the SDA line is low in the ninth clock period, an ACK takes place. If the SDA line is high in the ninth clock period, a scenario we call NACK occurs.

- When a master sends data to a slave, a NACK answered by the slave means that the communication has failed. The master needs to either generate a STOP to abort the current transfer or a START to restart the transfer.
- When a slave is transferring to a master, a NACK answered by the master means that the master sends a stop bit to terminate the communication after the current byte is transferred.

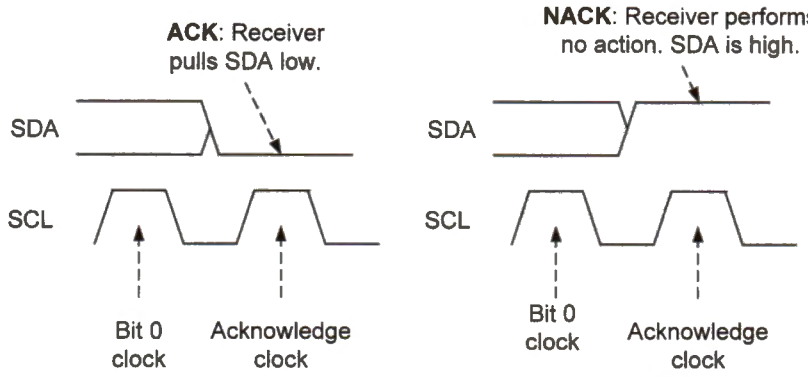


Figure 22-16. Comparison of ACK and NACK

Although the transfer clock signal is generated by the master, the slave can also control the transfer speed indirectly via ***clock stretching***. If the slave is too busy to receive another byte, it holds the SCL line low to force the master to wait. Due to the wire AND logic, the master cannot drive SCL high if the slave holds SCL low. The master resumes data transfer after the slave releases SCL.

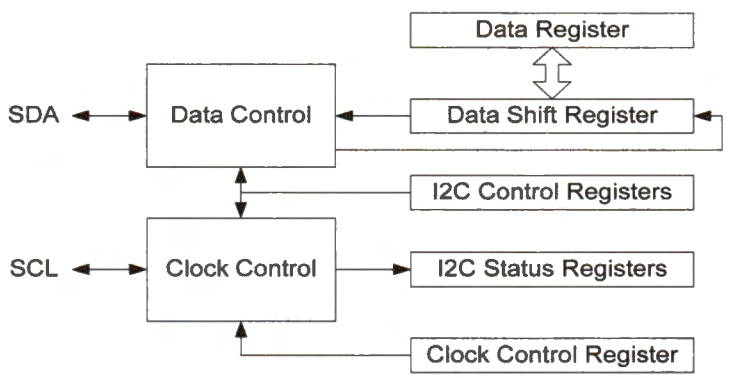


Figure 22-17. Simplified diagram of I²C

The STM32L processor has several I²C modules. Figure 22-17 shows the simplified data and clock control of one I²C module. The bytes stored in the data register are shifted in or out to the SDA line through an internal data shift register, with the most significant bit (MSB) in or out first.

- When the master is transmitting a byte to a slave, the master's I²C hardware automatically sets the TxE (transmitter buffer empty) flag in the status register if the master has received an acknowledge pulse (ACK) from the slave. A TxE event can inform software to send the next byte.
- On the other hand, if the master has received a byte successfully, the master's I²C hardware then automatically sets the RxNE (receiver buffer not empty) flag in the status register. A RxNE event can inform software to read the received byte.

If I²C interrupts are enabled, an I²C interrupt takes place under the following conditions.

- An I²C master generates an interrupt if a start bit is sent, a slave address is sent, hardware sets the TxE or RxNE flag, or the transfer of all data bytes completes.
- An I²C slave generates an interrupt if the address received matches its own address, a stop bit is received, or hardware sets the TxE or RxNE flag.

When there are multiple master devices on the I²C bus, clock synchronization and bus arbitration are required. During idle, both SCL and SDA have a high voltage level.

- ***Clock synchronization***. The SCL interface of all devices performs a logic AND operation. When one master pulls SCL low, no other masters can pull it high.
- ***Bus arbitration***. On the SCL rising edge, each master checks whether the SDA voltage level matches what it has sent. Whenever a master tries to transmit a high but detects a low on SDA, this master loses the arbitration. Each losing master immediately switches to slave receive mode because the winning master may be addressing it. A losing master will restart the transfer after it detects a STOP bit.

22.2.3 I²C Data Frame

Table 22-3 and Table 22-4 gives example message protocols between a master and a slave with a 7-bit and 10-bit address, respectively. If the address of a slave has 7 bits, the following lists the basic procedures.

1. The master begins by sending a start bit, the slave address, and a single bit (R/$\overline{W}$) representing the data transfer direction. The slave whose address matches the address sent by the master will answer with an ACK bit. The data transfer direction is represented by a single bit (R/$\overline{W}$). If the R/$\overline{W}$ bit is 1, then the master requests to receive data from the slave. If the R/$\overline{W}$ bit is 0, then the master requests to send data to the slave.
2. After the slave acknowledges the addressing successfully, data transfer takes place in the direction specified by the R/$\overline{W}$ bit. Data are transferred byte by byte. Each byte is followed by an ACK or NACK bit. Therefore, it takes 9 cycles to transfer one byte.

3. The master completes the communication by sending a STOP bit to the slave. The master may generate a START bit without sending a STOP bit first. This allows the master to communication with another slave or the same slave without releasing the bus. For example, a master may send a command to a slave and then immediately receive data from the slave.

S	Slave Address	R/$\overline{W}$	A	Data	A	Data	A	P
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

Table 22-3. Example of reading two bytes from or writing two bytes to an I²C slave with a 7-bit address (S = Start, P = Stop, A = Acknowledge, R/$\overline{W}$ = 1 for reading and 0 for writing)

S	Slave Address (higher 2 bits)	R/$\overline{W}$	A1	Slave Address (lower 8 bits)	A2	Data	A	Data	A	P
1 bit	7 bits (11110xx)	1 bit (0)	1 bit	8 bits	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

Table 22-4. Example of writing two bytes to an I²C slave with a 10-bit address. The slave address is in the first two bytes after the start bit. (S = Start, P = Stop, A = Acknowledge)

S	Slave Address (higher 2 bits)	R/$\overline{W}$	A1	Slave Address (lower 8 bits)	A2	S	Slave Address (higher 2 bits)	R/$\overline{W}$	A3	Data	A	Data	A	P
1 bit	7 bits (11110xx)	1 bit (0)	1 bit	8 bits	1 bit	1 bit	7 bits (11110xx)	1 bit (0)	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

Table 22-5. Example of reading two bytes from an I²C slave with a 10-bit address. After sending two address bytes (R/$\overline{W}$ = 0), the master repeats the start bit and sends again the first address byte, with R/$\overline{W}$ being 1 to indicate reading.

If the address of a slave has 10 bits, the communication protocol differs slightly.

- After the start bit, the first byte sent out by the master consists of a five-bit constant (11110), two most significant bits of the slave address, and the R/$\overline{W}$ bit. The master then sends the next byte, which is the least significant eight bits of the slave address.
- After the master sends out the first byte, all slave devices compare it to their own address. It is possible that more than one device finds a match between the two leading bits of the 10-bit address. Therefore, multiple ACK bits might be generated during the A1 clock period.
- After the master sends out the lower 8 bits of the slave address, at most one device finds an address match, and thus no multiple ACK bits are generated during the A2 clock period.
- As shown in Table 24-4 and Table 24-5, reading from or writing to a slave have different communication sequences.

22.2.4 Interfacing Serial Digital Thermal Sensors via I²C

This section gives an example in which the microprocessor interacts with multiple simple digital sensors via the I²C bus. In this example, we use TC74 digital thermal sensors from Microchip© Technology, which provide low-power 8-bit temperature measurements, with a resolution of $\pm 1°C$ and a conversion rate of 8 samples per second. The sensor has five pins, as listed in Table 22-6, supporting an I²C interface.

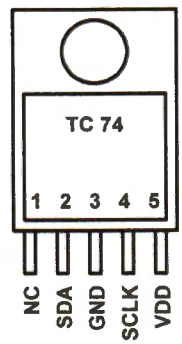


Figure 22-18. TC74 temperature sensor

Pin No.	Symbol	Type	Description
1	NC	None	No internal connection
2	SDA	Bidirectional	I²C serial data line
3	GND	Ground	System ground
4	SCL	Input	I²C serial clock line
5	Vdd	Power	Power supply input

Table 22-6. Pin connection of TC74 digital temperature sensor

The slave address of a TC74 sensor has 7 bits. There are eight different binary addresses (from `1001000` to `1001111`), depending on the part number of the device. For example, the default address of TC74 A5 is `1001101`. Software can change the address to one of the eight allowed addresses. This programmable address is helpful particularly when there are multiple TC74 sensors connected on the same I²C bus.

7	6	5	4	3	2	1	0
Read/Write	Read-only	Reserved					
1 = standby 0 = normal	1 = ready 0 = not ready						

Table 22-7. The configuration register of TC74 temperature sensor

TC74 internally has two 8-bit registers: (1) a temperature register with an address of `0x00`, and (2) a configuration register with an address of `0x01`.

- The temperature register holds the temperature measurement in units of degrees Celsius, represented in two's complement binary format. For example, a reading of `0x00` represents $0°C$ and a reading of `0xFF` represents $-1°C$. The representable temperature ranges from -65°C to 127°C.
- The configuration register is shown in Table 22-7. The most significant bit of the configuration register sets the sensor to either the standby state or the normal state. Bit 6 indicates whether the temperature data is ready or not. The tailing six bits are reserved.

The sensor provides two commands: (1) command code `0x00` to read the temperature register, and (2) command code `0x01` to read or write the configuration register. The following shows the communication sequence of reading and writing a byte from/to a TC74 temperature sensor.

Writing a Byte to TC74

S	Address	R/$\overline{W}$	ACK	Command	ACK	Data	ACK	P
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

- The R/$\overline{W}$ bit is `0` for writing data to the configuration register of the sensor.
- The command byte selects which register the data is written to.
- The data byte specifies the data content to be written to the target register.
- The `ACK` bit is sent by the sensor to the microcontroller to acknowledge the receipt of each byte.

Assume the 7-bit slave address of TC74 is `1001101`. Figure 22-19 shows the I²C communication sequence that puts TC74 in standby mode.

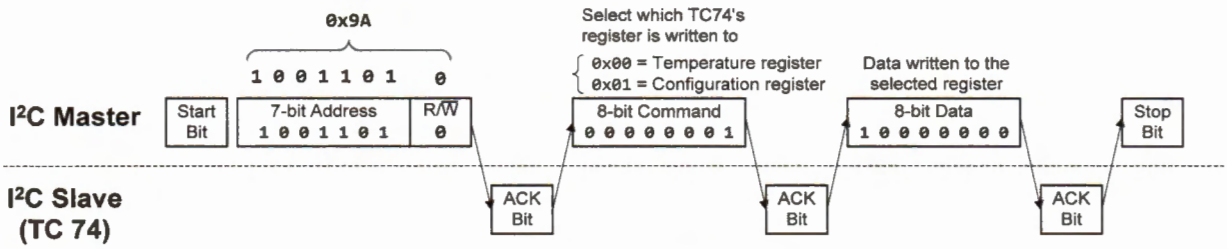


Figure 22-19. Putting TC74 with an address of `100110` (`0x4D`) in standby mode

Reading a Byte from TC74

Reading from TC74 involves two phases. Each phase has its own start bit, the address bits, and the $R/\overline{W}$ bit. The reason is that the second phase changes the transfer flow direction. This example shows that the master takes full control of all transfers. In the second phase, the master must re-initiate the transfer.

S	Address	R/$\overline{W}$	ACK	Command	ACK	S	Address	R/$\overline{W}$	ACK	Data	NACK	P
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	1 bit
	0x9A						0x9B					
1st phase: Select a register to read						2nd phase: Read the selected register						

- The first R/$\overline{W}$ bit is `0`, indicating that the master will write an 8-bit command. The command selects which register is to be read. It is either `0x00` or `0x01`.
- The second R/$\overline{W}$ bit is `1`, indicating that the master will read 8-bit data.
- The sensor sends the `ACK` bit to acknowledge the receipt of each byte.
- The master sends `NACK` to inform the slave to stop.

22.2.5 I²C Programmable Timings

Both I²C masters and slaves must meet the timing requirement of electrical signals defined in the I²C standard. Table 22-8 gives a few examples.

Timing Parameters	Standard		Fast		High-speed		Unit
	Min	Max	Min	Max	Min	Max	
SCL clock frequency	0	100	0	400	0	1000	kHz
Rise time of SDA & SCL	-	1000	20	300	-	120	µs
Fall time of SDA & SCL	-	300	20	300	-	120	µs
Low time of SCL	4.7	-	1.3	-	0.50	-	µs
High time of SCL	4.0	-	0.6	-	0.25	-	µs
Data hold time	5.0						µs
Data setup time	250	-	100	-	50	-	µs

Table 22-8. Example timing requirements of I²C electrical signals

Software can program the I²C's timing register (`TIMINGR`) to meet timing specifications. The timing register holds five parameters, including

- the clock prescaler (`PRESC[3:0]`),
- the clock `SCL` high period counter (`SCLH[7:0]`),
- the clock `SCL` low period counter (`SCLL[7:0]`),
- the data setup time counter (`SCLDEL[3:0]`, also called SCL delay counter), and
- the data hold time counter (`SDADEL[3:0]`, also called SDA delay counter).

22.2.5.1 Rise time and fall time

The ***rise time*** describes how long it takes for a voltage signal to increase from a low value to a high value. The 30-70 rise time is the amount of time taken by a voltage signal to increase from 30% of its final value to 70% of its final value. Similarly, the 30-70 ***fall time*** is the time taken by a signal to decrease from 70% of the final value to 30% of its final value. For the same circuit, the fall time is typically shorter than the rise time.

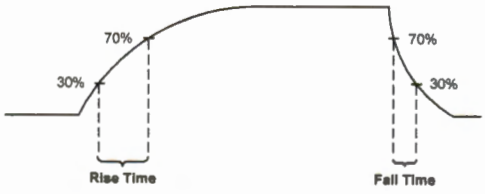


Figure 22-20. Definition of rise time and fall time

If two conductors are insulated and have a voltage potential between them, these two conductors form a capacitor. On circuit boards, the capacitance formed by copper traces, plates, wires, connections, and pins is called ***parasitic capacitance***. Although parasitic capacitance is very small, its impacts on high-frequency signals cannot be ignored.

For the I²C bus, the rise time is determined by the value of pull-up resistor R and the bus capacitance C. The bus capacitance is the sum of the parasitic capacitances of the wires, connectors, and pins. The maximum bus capacitance allowed for I²C is 400pF, which limits the maximum number of I²C devices connected to the same bus. Figure 22-21 shows the equivalent circuit of an I²C bus.

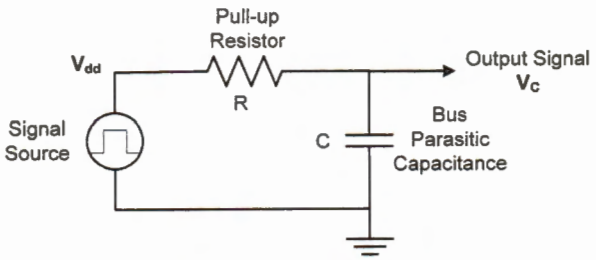
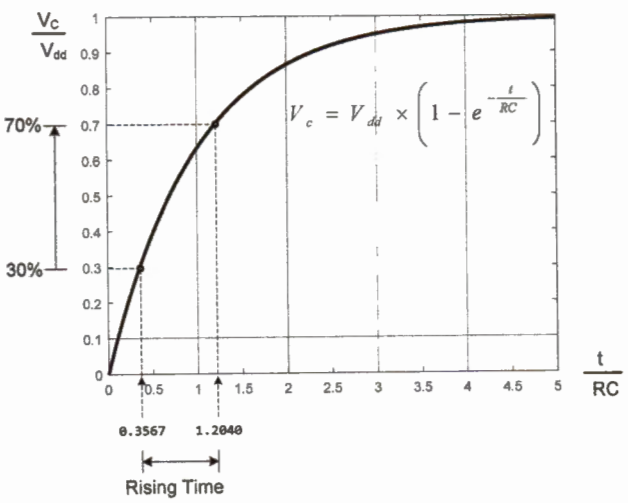


Figure 22-21. Equivalent circuit for I²C bus



$$\frac{V_c}{V_{dd}} = 1 - e^{-\frac{t}{RC}} = 0.30$$

Therefore, $t = 0.3567 \times RC$

$$\frac{V_c}{V_{dd}} = 1 - e^{-\frac{t}{RC}} = 0.70$$

Therefore, $t = 1.2040 \times RC$

$$\begin{aligned} Rise\ Time &= 1.2040 \times RC \\ &\quad - 0.3567 \times RC \\ &= 0.8473 \times RC \end{aligned}$$

Figure 22-22. Calculating the rise time

Figure 22-22 shows the calculation of the rise time. The pull-up resistors must be carefully chosen so that the rise time is smaller than the maximum allowed rise time, and the sink current is smaller than the maximum current allowed by an I²C pin.

$$\begin{cases} Rise\ Time = 0.8743 \times RC \leq RiseTime_{max} \\ \dfrac{V_{dd}}{R} \leq SinkCurrent_{max} \end{cases}$$

Therefore, we have

$$\frac{V_{dd}}{SinkCurrent_{max}} \leq R \leq \frac{RiseTime_{max}}{0.8743\ C}$$

22.2.5.2 Data hold time

When the falling edge of SCL is detected internally, a delay is inserted before data in the I2C TXDR register is sent out via the SDA line. The inserted delay t_{SDADEL} is programmed in the SDADEL field of register TIMINGR. The SDADEL counter starts to decrement automatically after the falling edge of SCL is detected internally. The delay completes when the SDADEL counter reaches zero.

All input signals are required to pass through analog and digital filters and slope detection, causing some delay. Suppose it takes t_{SYNC1} for the internal edge detector to detect a falling edge. As shown in Figure 22-23, the programmable SDA delay t_{SDADEL} is

$$t_{SDADEL} = [SDADEL \times (1 + PRESC) + 1] \times t_{I2C\_CLK}$$

As introduced earlier, the data line SDA is sampled periodically when the clock SCL is high, and SDA can change when SCL is low. The data hold time is defined as the amount of time after the falling edge of SCL during which SDA must remain at its current voltage level. Failing to do so may lead to SDA being improperly sampled when SCL transitions from high to low.

If t_r is the rise time, the ***data hold time*** is defined as follows:

$$Data\ Hold\ Time = t_{SYNC1} + t_{SDADEL} - t_r$$

The I²C standard specifies the minimum data hold time based on the speed mode. Because we have the following relationship:

$$Data\ Hold\ Time > t_{SDADEL}$$

we can program the SDADEL counter in the TIMINGR register such that

$$t_{SDADEL} > Minimum\ Data\ Hold\ Time\ Specified$$

Thus, the actual data hold time is guaranteed to be larger than the requirement specified.

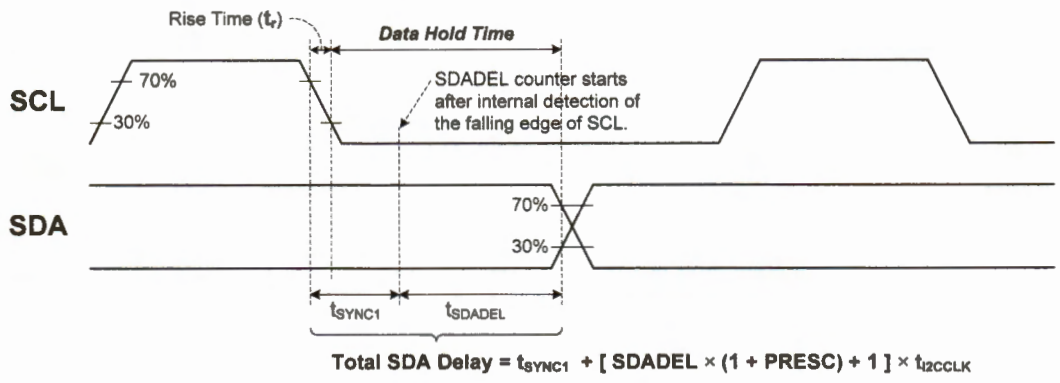


Figure 22-23. I²C data hold time.

22.2.5.3 Data setup time

When the clock line SCL is low, the data line SDA is updated either by the master or the slave, depending on the transfer direction. The data line is periodically sampled once the clock line SCL becomes high. Thus, SDA must remain its current voltage level before sampling starts, *i.e.*, before the rising edge of the clock takes place.

The ***data setup time*** is defined as the amount of time SCL is held low after a data bit has been placed on SDA.

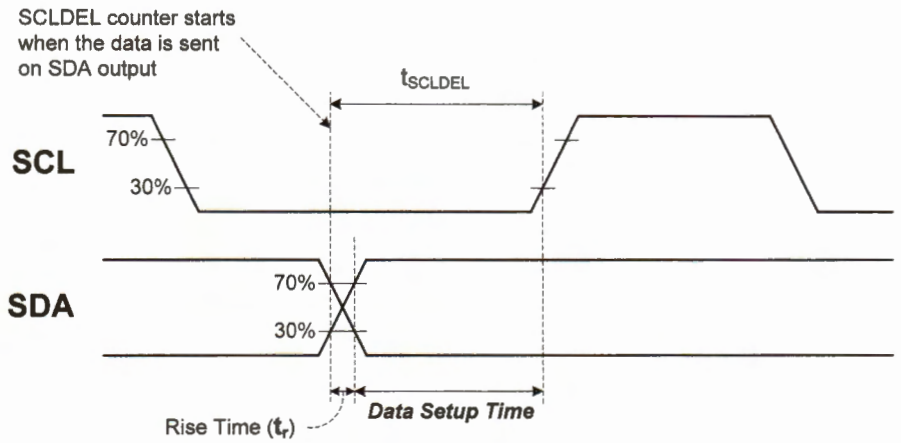


Figure 22-24. I²C data setup time

Software can configure the SCLDEL counter in the TIMINGR register such that the programmable SCL delay t_{SCLDEL} meets the following requirement:

$$t_{SCLDEL} > Minimum\ Data\ Setup\ Time\ Specified$$

The SCLDEL counter starts to decrement automatically after a data bit has been placed on SDA. In addition, we have

$$t_{SCLDEL} = (SCLDEL + 1) \times (1 + PRESC) \times t_{I2CCLK}$$

22.2.5.4 Master clock's minimum high and low time

The I²C clock timing is programmed by the SCLL and SCLH fields in the timing register. These two counters set the clock's low- and high-level durations, as well as the clock period, as shown below:

$$t_{low} = t_{SYNC1} + (SCLL + 1) \times (1 + PRESC) \times t_{I2CCLK}$$

$$t_{high} = t_{SYNC2} + (SCLH + 1) \times (1 + PRESC) \times t_{I2CCLK}$$

$$t_{period} = t_{low} + t_{high}$$

where t_{SYNC1} and t_{SYNC2} are the time it takes for the internal edge detector to detect a falling edge and a rising edge respectively. These delays include the delay caused by the analog filter, the digital filter, and the hardware edge detector.

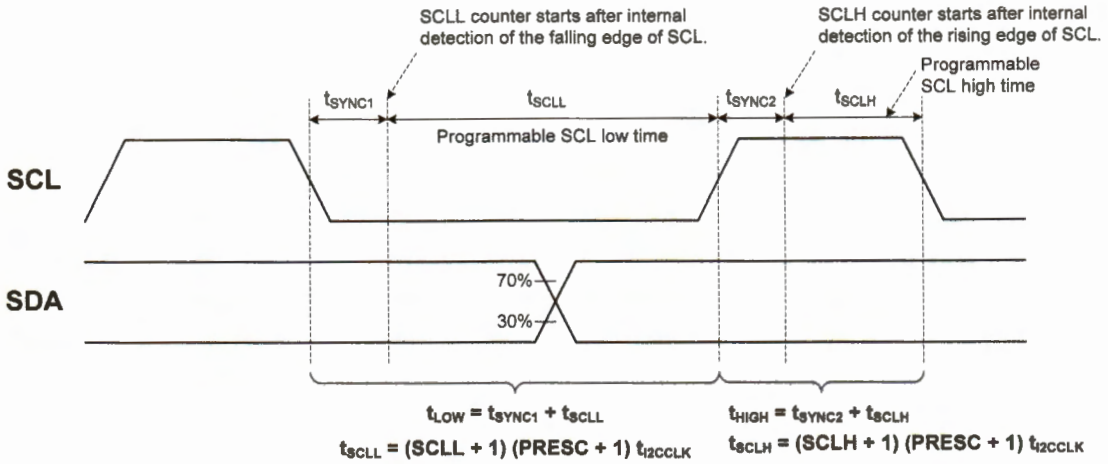


Figure 22-25. I2C clock high and low durations

22.2.5.5 Example of setting the I²C timing

Table 22-9 lists key parameters of TC74 sensors.

Parameters	Value
Min high clock period	4.0 μs
Min low clock period	4.7 μs
Min data hold time	1250 ns
Min data setup time	1000 ns
Max rise time	1000 ns
Max fall time	300 ns
Input Capacitance SDA and SCL	5.0 pF
Maximum current on any pin	±50 mA

Table 22-9. Electrical parameters of TC74 temperature sensor

Rise time: if we use 1KΩ to pull up the SDA and SCL

$$t_{rise} = 0.8743 \times RC = 0.8742 \times 1000\Omega \times 5.0 \times 10^{-12}F = 4.4\,ns < 1000\,ns$$

Fall time: the fall time is the same as the rise time.

$$t_{fall} = t_{rise} = 4.4\,ns < 300\,ns$$

Electric Current:

$$Current = \frac{3V}{1000\Omega} = 0.3\text{ mA} < 50\text{ mA}$$

Let's use I2C1 in our example of setting up the timing register (TIMINGR) to meet the timing requirements. Suppose the system clock (SYSCLK) has been set to 80 MHz. Example 22-11 selects the SYSCLK as the source clock to drive I2C1.

```
RCC->APB1ENR1 |= RCC_APB1ENR1_I2C1EN;        // I2C1 clock enable

// 00 = PCLK, 01 = SYSCLK, 10 = HSI16, 11 = Reserved
RCC->CCIPR &= ~RCC_CCIPR_I2C1SEL;            // Clear bits
RCC->CCIPR |=  RCC_CCIPR_I2C1SEL_0;          // Select SYSCLK

RCC->APB1RSTR1 |=  RCC_APB1RSTR1_I2C1RST;    // 1 = Reset I2C1
RCC->APB1RSTR1 &= ~RCC_APB1RSTR1_I2C1RST;    // Complete the reset
```

Example 22-11. Selecting SYSCLK to the clock to drive I2C1

Suppose we choose the clock prescaler (PRESC) as 7. Then, the clock frequency of I2C1 is:

$$f_{I2CCLK} = \frac{f_{SYSCLK}}{1 + PRESC} = \frac{80\ MHz}{1+7} = 10\ MHz$$

Therefore,

$$t_{I2C\_PRESC} = \frac{1}{f_{I2CCLK}} = \frac{1}{10MHz} = 0.1\ \mu s$$

Data setup time: We select SCLDEL as 14.

$$t_{setup} > (1 + SCLDEL) \times t_{I2C\_PRESC} = (1 + 14) \times 0.1\ \mu s = 1.5\ \mu s > 1.0\ \mu s$$

Data hold time: Suppose we select SDADEL as 15.

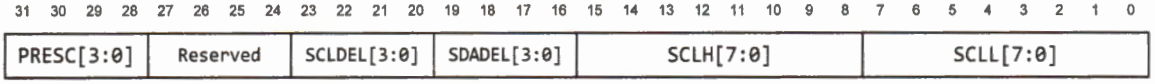
$$t_{hold} > t_{SDADEL} > (1 + SDADEL) \times t_{I2C\_PRESC} = (1 + 15) \times 0.1\ \mu s = 1.6\ \mu s > 1.25\ \mu s$$

Low clock period: We select SCLL as 49.

$$t_{low} > (SCLL + 1) \times t_{I2C\_PRESC} = (1 + 49) \times 0.1\ \mu s = 5.0\ \mu s > 4.7\ \mu s$$

High clock period: We select SCLH as 49, too.

$$t_{high} > (SCLH + 1) \times t_{I2C\_PRESC} = (1 + 49) \times 0.1\ \mu s = 5.0\ \mu s > 4.0\ \mu s$$



In sum, software can program the timing register (TIMINGR) as follows:

```
I2C1->TIMINGR = 7U << 28 | 14U << 20 | 15U << 16 | 49U << 8 | 49U;
```

22.2.6 Sending Data to I²C Slave via Polling

During idle time, both SCL and SDA lines are pulled high. When the master wants to send data to a slave, the master first wait until the bus is ready by checking the BUSY flag in the ISR register, then sends a start bit by pulling SDA low, and places the clock signal on SCL. Then, the master sends the address frame, with the least significant bit being 0 to indicate that the master is the transmitter. After the address frame, the master starts to transfer data byte by byte. For each transfer, software must wait until the TXIS flag or the NACK flag is set. Hardware automatically sets the TXIS flag after it receives the acknowledgment bit from the slave, and clears the TXIS flag when the byte to be transferred has been written to the transmit data register (TXDR). The master stops the data transfer by terminating the clock on SCL and pulling SDA high. Software must wait until hardware sets the transfer complete flag (TC).

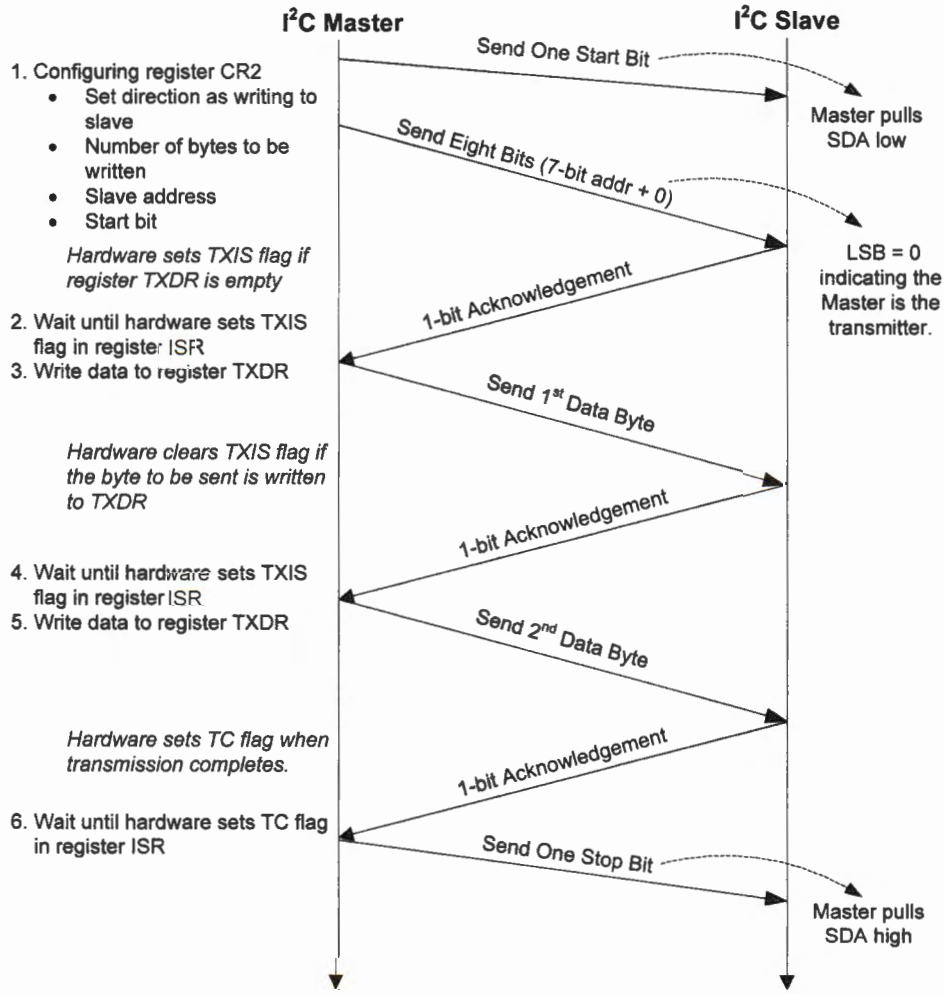


Figure 22-26. Time diagram when a master sends two bytes to a slave

22.2.7 Receiving Data from I²C Slave via Polling

After the master configures control register 2 (CR2), the master sends one start bit and the address byte on the SCL bus. The target slave responds with an acknowledgment bit (ACK) and then starts to transfer the data to the master byte by byte. When the master receives a byte, hardware sets the RXNE flag. Before software can read the receive data register (RXDR), software must wait until the RXNE flag is set. Hardware automatically clears the RXNE flag when software reads RXDR. If auto-end mode is used, the master will automatically send a NACK and a stop bit after the last byte has been transferred. Otherwise, software must send the stop bit explicitly. Additionally, software should wait until hardware sets the transfer complete flag (TC).

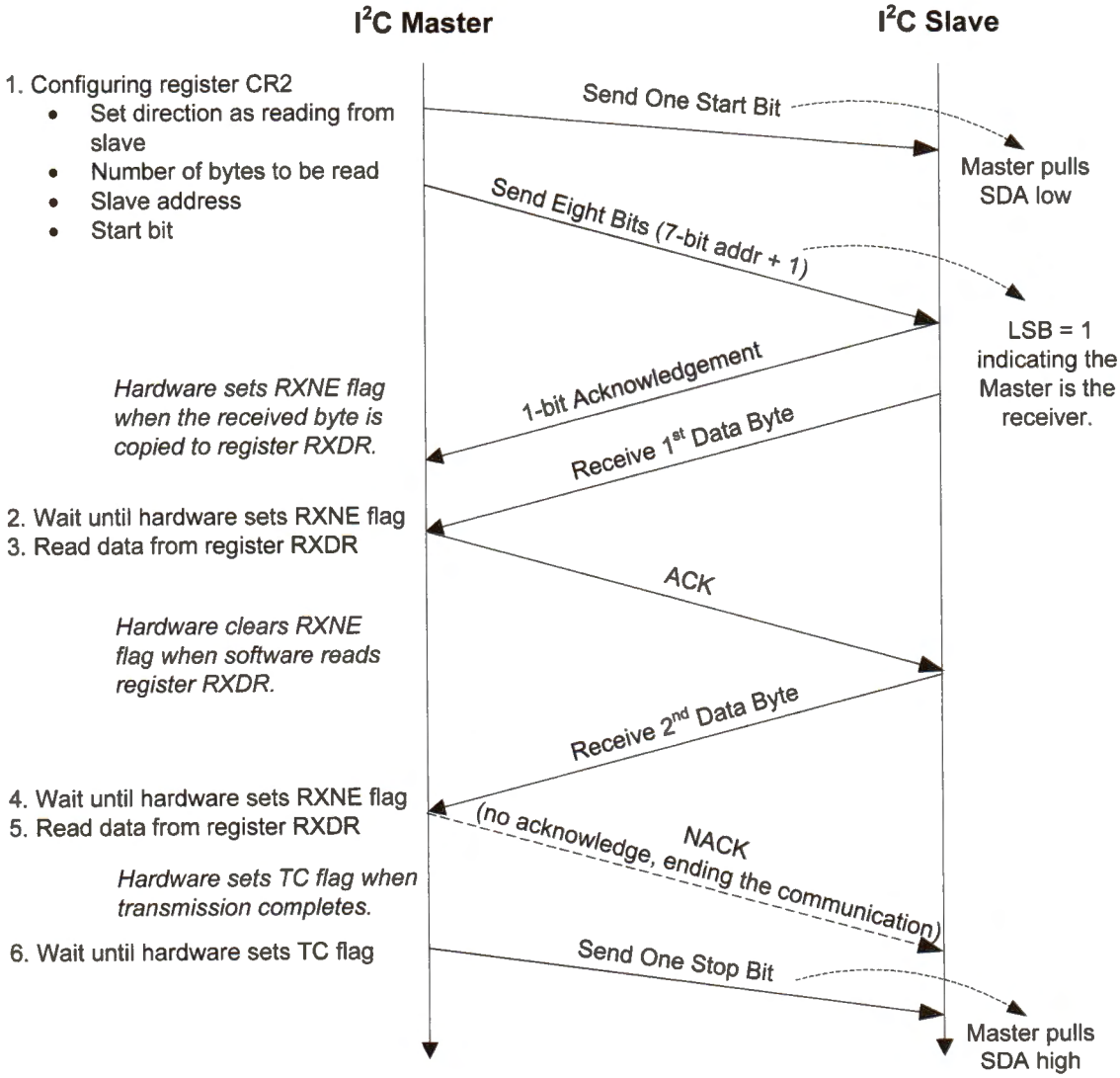


Figure 22-27. Time diagram when the master receives two bytes from a slave

22.2.8 Interfacing a Temperature Sensor via Polling

This section shows a C example program that interfaces with TC74 digital temperature sensors. Make sure that the VDD of the TC74 temperature sensor is 5V, as shown in Figure 22-28.

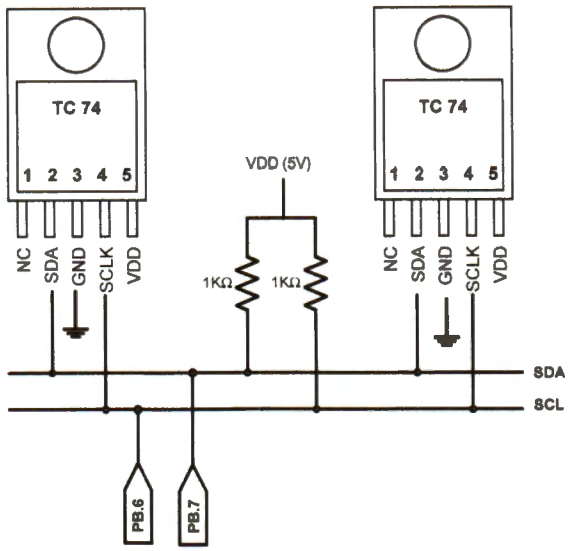


Figure 22-28. Connecting two TC74 digital temperature sensors via the I²C bus.

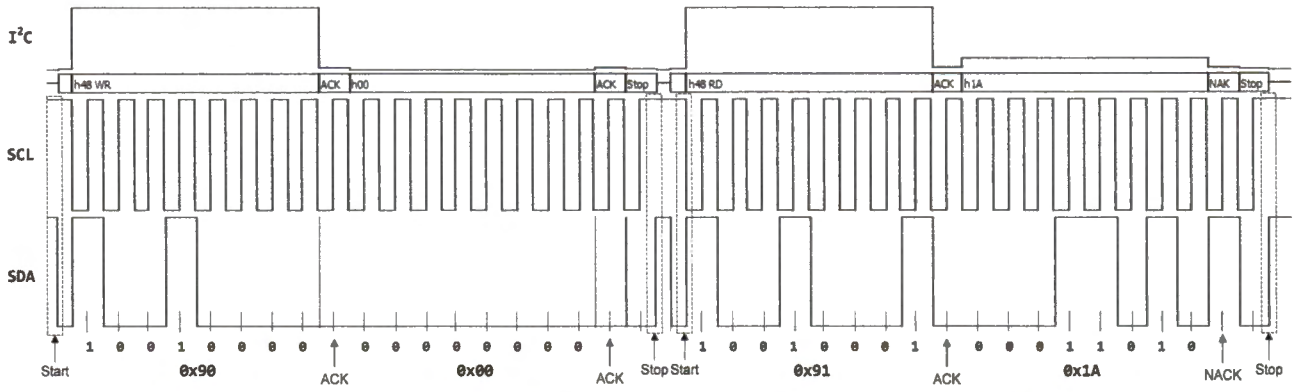


Figure 22-29. Signal capture on the SDA (the top signal) and SCL pin (the bottom signal).

There are two I²C modules on the STM32L processors. In this example, we use the first I²C module, which consists of the following two pins.

Pin	Connection	Mode	AF	Output Type	Pull-up/Pull-down	Clock
PB.6	I2C1_SCL	AF	I2C1	Open-drain	Pull-up	40 MHz
PB.7	I2C1_SDA	AF	I2C1	Open-drain	Pull-up	40 MHz

The following presents the initialization of the first I²C module.

```
void I2C_Init (I2C_TypeDef * I2Cx) {

  uint32_t OwnAddr = 0x52;

  // Enable the I2C clock & Select SYSCLK as the clock source
  // See Example 22-11
  ...

  // I2C CR1 Configuration
  // When the I2C is disabled (PE=0), the I2C performs a software reset.
  I2Cx->CR1 &= ~I2C_CR1_PE;          // Disable I2C
  I2Cx->CR1 &= ~I2C_CR1_ANFOFF;      // 0: Analog noise filter enabled
  I2Cx->CR1 &= ~I2C_CR1_DNF;         // 0000: Digital filter disabled
  I2Cx->CR1 |= I2C_CR1_ERRIE;        // Errors interrupt enable
  I2Cx->CR1 &= ~I2C_CR1_SMBUS;       // SMBus Mode: 0 = I2C mode;  1 = SMBus mode
  I2Cx->CR1 &= ~I2C_CR1_NOSTRETCH; // Enable clock stretching

  // I2C TIMINGR Configuration (See Section 22.2.5.5)
  I2Cx->TIMINGR = 0;
  // SysTimer = 80 MHz, PRESC = 7, 80MHz/(1 + 7) = 10 MHz
  I2Cx->TIMINGR &= ~I2C_TIMINGR_PRESC; // Clear the prescaler
  I2Cx->TIMINGR |= 7U << 28;           // Set clock prescaler to 7
  I2Cx->TIMINGR |= 49U;          // SCLL: SCL low period (master mode) > 4.7 us
  I2Cx->TIMINGR |= 49U << 8;     // SCLH: SCL high period (master mode) > 4.0 us
  I2Cx->TIMINGR |= 14U << 20;  // SCLDEL: Data setup time > 1.0 us
  I2Cx->TIMINGR |= 15U << 16;  // SDADEL: Data hold time > 1.25 us

  // I2C Own address 1 register (I2C_OAR1)
  I2Cx->OAR1 &= ~I2C_OAR1_OA1EN;
  I2Cx->OAR1 = I2C_OAR1_OA1EN | OwnAddr; // 7-bit own address
  I2Cx->OAR1 &= ~I2C_OAR2_OA2EN;         // Disable own address 2

  // I2C CR2 Configuration
  I2Cx->CR2 &= ~I2C_CR2_ADD10;  // 0 = 7-bit mode, 1 = 10-bit mode
  I2Cx->CR2 |= I2C_CR2_AUTOEND; // Enable the auto end
  I2Cx->CR2 |= I2C_CR2_NACK;    // For slave mode: set NACK
  I2Cx->CR1 |= I2C_CR1_PE;      // Enable I2C1
}
```

Example 22-12. Initializing I²C

The following subroutine *I2C_Start*() generates a start bit. This subroutine mainly programs the control register CR2. It selects the automatic end mode, in which a STOP bit is sent automatically when all bytes have been transferred. Hardware clears the STOP flag when a STOP bit has been sent successfully.

Similarly, the START bit is set by software and is cleared by hardware after the START bit followed by the slave address is sent. The input argument Direction selects the data transfer direction.

```
void I2C_Start(I2C_TypeDef * I2Cx, uint32_t DevAddress,
                        uint8_t Size, uint8_t Direction) {

  // Direction = 0: Master requests a write transfer
  // Direction = 1: Master requests a read transfer

  uint32_t tmpreg = I2Cx->CR2;
  tmpreg &= (uint32_t)~((uint32_t)(I2C_CR2_SADD    | I2C_CR2_NBYTES   |
                                   I2C_CR2_RELOAD  | I2C_CR2_AUTOEND |
                                   I2C_CR2_RD_WRN  | I2C_CR2_START   |
                                   I2C_CR2_STOP));

  if (Direction == READ_FROM_SLAVE)
    tmpreg |= I2C_CR2_RD_WRN;    // Read from Slave
  else
    tmpreg &= ~I2C_CR2_RD_WRN;  // Write to Slave

  tmpreg |= (uint32_t)(((uint32_t) DevAddress  & I2C_CR2_SADD) |
                       (((uint32_t) Size << 16) & I2C_CR2_NBYTES));

  tmpreg |= I2C_CR2_START;
  I2Cx->CR2 = tmpreg;
}
```

Example 22-13. Sending the start bit

The following subroutine generates a STOP bit. Hardware sets the STOPF flag after the STOP bit has been detected.

```
void I2C_Stop(I2C_TypeDef * I2Cx){

  // Master: Generate STOP bit after the current byte has been transferred
  I2Cx->CR2 |= I2C_CR2_STOP;

  // Wait until STOPF flag is reset
  while( (I2Cx->ISR & I2C_ISR_STOPF) == 0 );

  I2Cx->ICR |= I2C_ICR_STOPCF; // Write 1 to clear STOPF flag
}
```

Example 22-14. Generating a STOP bit

The bus busy flag in the ISR register indicates whether a data transfer is currently taking place. Hardware sets this flag once a START bit is detected. Hardware clears this flag when a STOP bit is detected.

```
void I2C_WaitLineIdle(I2C_TypeDef * I2Cx){
  // Wait until I2C bus is ready
  while( (I2Cx->ISR & I2C_ISR_BUSY) == I2C_ISR_BUSY ); // If busy, wait
}
```

Example 22-15. Waiting for the line idle

The *I2C_SendData*() sends multiple bytes to a target slave.

- It first waits until the SDA and SCL lines are idle (*i.e.*, the voltage of both lines are high). Then it sends a start bit and the slave address.
- Next, it begins to send out the data byte by byte. When the master successfully sends a byte to the target slave, the transmitter register empty (TXIS) flag is set by hardware. Before sending the next byte, normally the master should wait until TC is set. Writing to the data register (TXDR) clears the TXIS flag automatically.
- After all bytes have been sent, hardware sets the TC flag. Software waits until the TC flag is set. Hardware automatically clears the TC flag when the START bit or the STOP bit in the control register CR2 is set.
- At the end, the subroutine sends a stop bit and waits until the SDA and SCL lines are idle.

```
int8_t I2C_SendData(I2C_TypeDef *I2Cx,  uint8_t SlaveAddress,
                    uint8_t *pData, uint8_t Size) {
  int i;
  if (Size <= 0 || pData == NULL) return -1;

  // Wait until the line is idle
  I2C_WaitLineIdle(I2Cx);

  // The last argument: 0 = Sending data to the slave
  I2C_Start(I2Cx, SlaveAddress, Size, 0);

  for (i = 0; i < Size; i++) {
    // TXIS bit is set by hardware when the TXDR register is empty and the
    // data to be transmitted must be written in the TXDR register. It is
    // cleared when the next data to be sent is written in the TXDR register.
    // The TXIS flag is not set when a NACK is received.
    while( (I2Cx->ISR & I2C_ISR_TXIS) == 0 );

    // TXIS is cleared by writing to the TXDR register
    I2Cx->TXDR = pData[i] & I2C_TXDR_TXDATA;
  }

  // Wait until TC flag is set
  while((I2Cx->ISR & I2C_ISR_TC) == 0 && (I2Cx->ISR & I2C_ISR_NACKF) == 0);

  if( (I2Cx->ISR & I2C_ISR_NACKF) != 0 )
    return -1;

  I2C_Stop(I2Cx);

  return 0;
}
```

Example 22-16. Transmitting data via I²C

The subroutine *I2C_ReceiveData*() receives data from an I²C slave. The program enables the acknowledgment in the subroutine *I2C_Start*(). Software waits until hardware sets the receive data register not empty flag (RXNE). Hardware sets the RXNE flag when the received data has been copied from the internal shift register into the receive data register (RXDR) and is ready to read. The RXNE flag is cleared when software reads register RXDR.

```
int8_t I2C_ReceiveData(I2C_TypeDef * I2Cx, uint8_t SlaveAddress,
                       uint8_t *pData, uint8_t Size) {
  int i;

  if (Size <= 0 || pData == NULL) return -1;

  I2C_WaitLineIdle(I2Cx);

  I2C_Start(I2Cx, SlaveAddress, Size, 1);  // 1 = Receiving from the slave

  for (i = 0; i < Size; i++) {
    // Wait until RXNE flag is set
    while( (I2Cx->ISR & I2C_ISR_RXNE) == 0 );
    pData[i] = I2Cx->RXDR & I2C_RXDR_RXDATA;
  }

  while((I2Cx->ISR & I2C_ISR_TC) == 0); // Wait until TCR flag is set

  I2C_Stop(I2Cx);
  return 0;
}
```

Example 22-17. Receiving data via I²C

The following program shows the software that reads the temperature from the temperature sensor TC74.

```
  uint8_t Data_Receive[6];
  uint8_t Data_Send[6];

  System_Clock_Init();        // Set System Clock to 80 MHz
  I2C_GPIO_init();
  I2C_Initialization(I2C1);

  while(1){
    SlaveAddress = 0x48<<1;  // A0 = 1001000 = 0x48
    Data_Send[0] = 0x00;     // 00 = command to read temperature register
    I2C_SendData(I2C1, SlaveAddress, Data_Send, 1);
    I2C_ReceiveData(I2C1, SlaveAddress, Data_Receive, 1);
    for(i = 0; i < 50000; i++); // Short software delay
  }
```

Example 22-18. Interfacing TC74 temperature sensor

22.2.9 Transferring Data via DMA on I²C Master

To enable DMA transfer, the *I2C_Init()* function given in Example 22-12 must add the following statements:

```
I2Cx->CR1 |= I2C_CR1_TXDMAEN;  // Enable DMA transmission requests
I2Cx->CR1 |= I2C_CR1_RXDMAEN;  // Enable DMA reception requests
```

In the following, we use I2C1 (PB6 and PB7) as an example to show DMA configuration on I2. As shown in Table 19-1, `I2C1_TX` and `I2C1_RX` can be connected to the channel 6 and 7 of the DMA controller 1, respectively. Example 22-19 and Example 22-20 show the DMA configuration. Note that the slave address cannot be transferred via DMA.

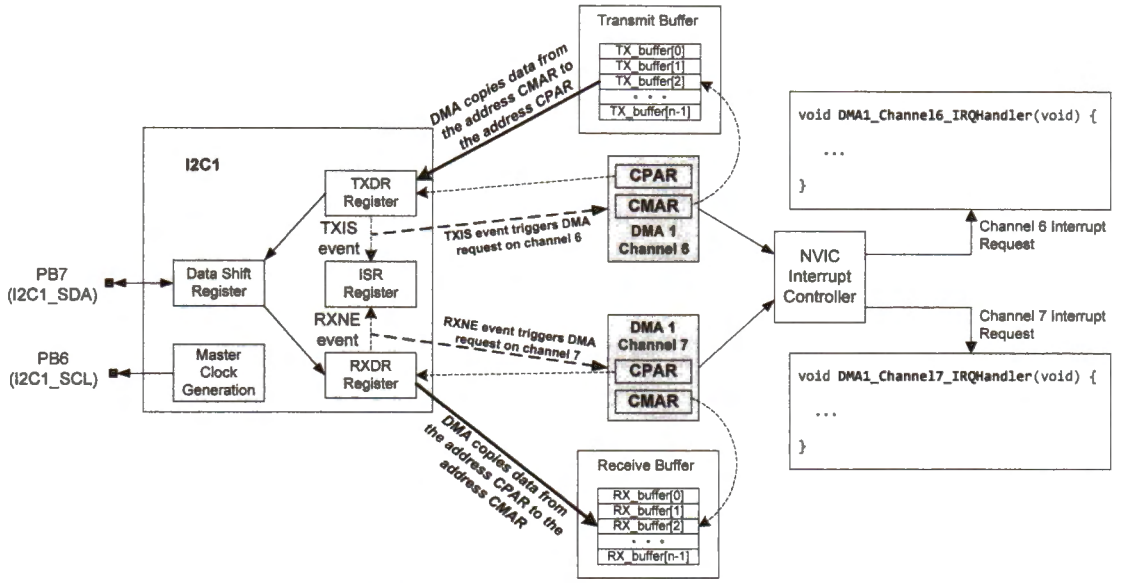


Figure 22-30. DMA configuration for I2C1 on STM32L4

```
void DMA_Configure_I2C_TX (uint8_t *pTxBuffer, uint32_t size) {
  RCC->AHB1ENR |= RCC_AHB1ENR_DMA1EN;      // Enable DMA1 clock
  // Connect I2C1_TXDR to DMA 1 Channel 6
  DMA1_Channel6->CCR &= ~DMA_CCR_EN;       // Disable DMA channel
  DMA1_Channel6->CCR &= ~DMA_CCR_MEM2MEM; // Disable memory to memory mode
  DMA1_Channel6->CCR &= ~DMA_CCR_PL;       // Channel priority level
  DMA1_Channel6->CCR |=  DMA_CCR_PL_1;     // Set DMA priority to high
  DMA1_Channel6->CCR &= ~DMA_CCR_PSIZE;    // Peripheral data size 00 = 8 bits
  DMA1_Channel6->CCR &= ~DMA_CCR_MSIZE;    // Memory data size: 00 = 8 bits
  DMA1_Channel6->CCR &= ~DMA_CCR_PINC;     // Disable peripheral increment mode
  DMA1_Channel6->CCR |=  DMA_CCR_MINC;     // Enable memory increment mode
  DMA1_Channel6->CCR &= ~DMA_CCR_CIRC;     // Disable circular mode
  DMA1_Channel6->CCR |=  DMA_CCR_DIR;      // Transfer direction: to peripheral
  DMA1_Channel6->CCR |=  DMA_CCR_TCIE;     // Transfer complete interrupt enable
  DMA1_Channel6->CCR &= ~DMA_CCR_HTIE;     // Disable Half transfer interrupt
  DMA1_Channel6->CNDTR = size;             // Number of data to transfer
```

```
  DMA1_Channel6->CPAR  = (uint32_t)&(I2C1->TXDR);  // Peripheral address
  DMA1_Channel6->CMAR  = (uint32_t) pTxBuffer;     // Transmit buffer address
  DMA1_CSELR->CSELR &= ~DMA_CSELR_C6S;     // See Table 19-1
  DMA1_CSELR->CSELR |= 3<<24;              // Map channel 6 to I2C1_TX
  DMA1_Channel6->CCR |= DMA_CCR_EN;        // Enable DMA channel
}
```

Example 22-19. Configuring DMA 1 channel 6 for I2C1 transmit

```
void DMA_Configure_I2C_RX (uint8_t *pRxBuffer, uint32_t size) {
  RCC->AHB1ENR |= RCC_AHB1ENR_DMA1EN;      // Enable DMA1 clock
  // Connect I2C1_RXDR to DMA 1 Channel 7
  DMA1_Channel7->CCR &= ~DMA_CCR_EN;       // Disable DMA channel
  DMA1_Channel7->CCR &= ~DMA_CCR_MEM2MEM; // Disable memory to memory mode
  DMA1_Channel7->CCR &= ~DMA_CCR_PL;       // Channel priority level
  DMA1_Channel7->CCR |=  DMA_CCR_PL_1;     // Set DMA priority to high
  DMA1_Channel7->CCR &= ~DMA_CCR_PSIZE;    // Peripheral data size 00 = 8 bits
  DMA1_Channel7->CCR &= ~DMA_CCR_MSIZE;    // Memory data size: 00 = 8 bits
  DMA1_Channel7->CCR &= ~DMA_CCR_PINC;     // Disable peripheral increment mode
  DMA1_Channel7->CCR |=  DMA_CCR_MINC;     // Enable memory increment mode
  DMA1_Channel7->CCR &= ~DMA_CCR_CIRC;     // Disable circular mode
  DMA1_Channel7->CCR |=  DMA_CCR_DIR;      // Transfer direction: to peripheral
  DMA1_Channel7->CCR |=  DMA_CCR_TCIE;     // Transfer complete interrupt enable
  DMA1_Channel7->CCR &= ~DMA_CCR_HTIE;     // Disable Half transfer interrupt
  DMA1_Channel7->CNDTR = size;             // Number of data to transfer
  DMA1_Channel7->CPAR  = (uint32_t)&( I2C1->RXDR);  // Peripheral address
  DMA1_Channel7->CMAR  = (uint32_t) pRxBuffer;      // Transmit buffer address
  DMA1_CSELR->CSELR &= ~DMA_CSELR_C7S;     // See Table 19-1
  DMA1_CSELR->CSELR |= 3<<20;              // Map channel 6 to I2C1_RX
  DMA1_Channel7->CCR |= DMA_CCR_EN;        // Enable DMA channel
}
```

Example 22-20. Configuring DMA 1 channel 7 for I2C1 reception

The following examples illustrates how to send or receive data via DMA.

```
void I2C_SendData(uint8_t *pTxBuffer, uint32_t size, uint8_t SlaveAddress) {
  // DMA must be initialized before setting the START bit
  DMA_Configure_I2C_TX(pTxBuffer, size);    // Configure DMA 1 Channel 6
  I2C_WaitLineIdle(I2C1);                   // Wait until I2C is available
  I2C_Start(I2C1, SlaveAddress, Size, 0);   // 0 = Sending to the slave
}
```

Example 22-21. Sending data to an I²C slave via DMA

```
void I2C_ReceiveData(uint8_t *pRxBuffer, uint32_t size,uint8_t SlaveAddress) {
  // DMA must be initialized before setting the START bit
  DMA_Configure_I2C_TX(pRxBuffer, size);    // Configure DMA 1 channel 7
  I2C_WaitLineIdle(I2C1);                   // Wait until I2C is available
  I2C_Start(I2C1, SlaveAddress, Size, 1);   // 1 = Receiving from the slave
}
```

Example 22-22. Receiving data from an I²C slave via DMA

22.3 Serial Peripheral Interface Bus (SPI)

Serial peripheral interface (SPI) is a synchronous serial communication interface widely used to exchange data between a microprocessor and peripheral devices using four wires. For example, a digital camera often uses SPI to control its lens and save photos to a MMC or SD media.

SPI is simple, has low power requirements, and supports high throughput. Disadvantages of SPI include that it does not support multiple masters, and slaves cannot start the communication or control data transfer speed. The master initiates and controls all communications.

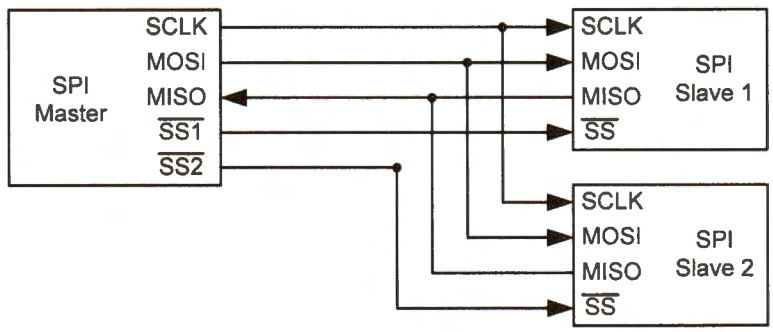


Figure 22-31. A SPI master device connecting to multiple SPI slave devices

A SPI interface consists of four lines: a master-in-slave-out data line (`MISO`), a master-out-slave-in data line (`MOSI`), a serial clock line (`SCLK`), and an active-low slave select line ($\overline{SS}$), as shown in Figure 22-31. SPI is also called four-wire serial interface.

SPI only supports a single master communicating with multiple slave devices. As shown in Figure 22-33, when the master wishes to exchange data with a slave, it pulls down the corresponding select line (SS_N). The master then generates clock pulses to coordinate the data transmission on the `MOSI` and `MISO` lines.

Data exchange can take place in both directions simultaneously, and this two-way serial channel is often called ***full duplex***. Data bits are transmitted on both the `MOSI` line and the `MISO` line synchronously, with the flow directions opposite to each other. Note the `SCLK` line has only one direction, and only the master can generate the clock signal. The slave devices cannot control the clock line.

When there are multiple slave devices, the master decides which slave device it wants to communicate. There is a dedicated Slave Select (`SS`) line for each slave device. The master

selects the target slave device by pulling the corresponding SS line to a low voltage prior to data transfer. The selected slave device then listens for the clock and MOSI signals. When there is only one slave device, the SS line can be directly connected to ground physically, or the program can make the slave continuously selected.

22.3.1 Data Exchange

SPI is a synchronous protocol, and the slave devices must send and receive data based on the clock provided by the master. It differs from an asynchronous protocol in which no clock signal is provided physically. SPI devices must exchange data at the same speed.

The master and a slave perform data exchange at synchronized time steps based on the clock signal generated by the master.

SPI master provides clock signal (SCLK) to SPI slaves.

- When a bit is shifted out on the MISO line from the slave's data register during a clock period, a new data bit is shifted into this register from the MOSI line in the same clock period, as shown in Figure 22-32.
- When one device writes a bit to the data line at the rising or falling edge of the clock, the other device then reads the bit at the opposite edge of the same clock period.
- The data transfer size is usually a byte or halfword (16 bits).

Communication from the master to a slave and communication from a slave to the master are always taking place concurrently. In each communication link (either MISO or MOSI), each device sends out a data item and at the same time receives a new data item. No devices can just be a transmitter or a receiver.

Therefore, when a slave wants to send data to the master via the MISO line, the slave must wait for the clock signal. At the same time, the master must send some dummy data out via the MOSI line to generate the clock signal to initiate the data transfer, as shown in Figure 22-32.

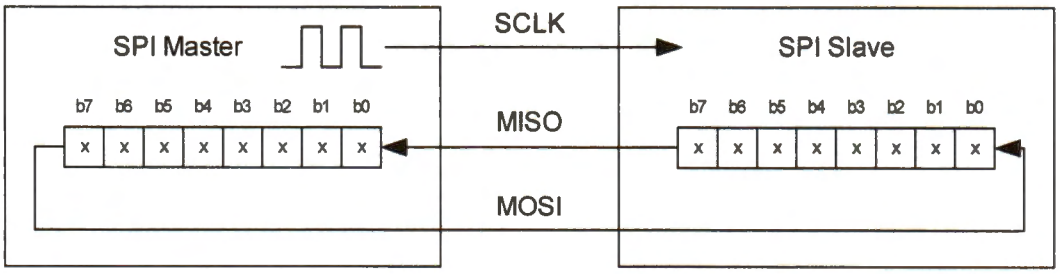


Figure 22-32. A byte is shifted out and in simultaneously via MOSI and MISO.

When a master exchanges data with slave *n*, the master must set $\overline{SS_n}$ low to select slave *n*, as shown in Figure 22-33. During the communication, the most significant bit of both data registers is sent out first.

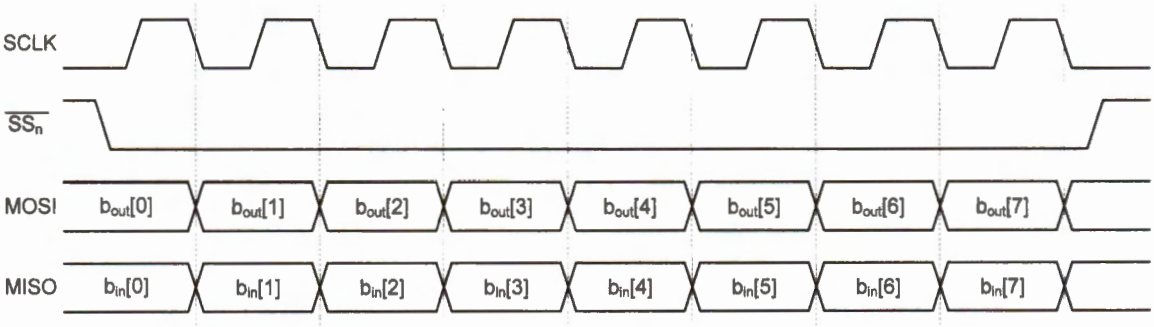


Figure 22-33. Communication signals between a master and slave *n*. In this example, the most significant bit is transferred first.

Figure 22-34 shows the signal of `SCLK` and `MOSI` when two bytes (`0xAA` and `0x3C`) are sent out. In this example, the least significant bit (LSB) of each data is sent out first.

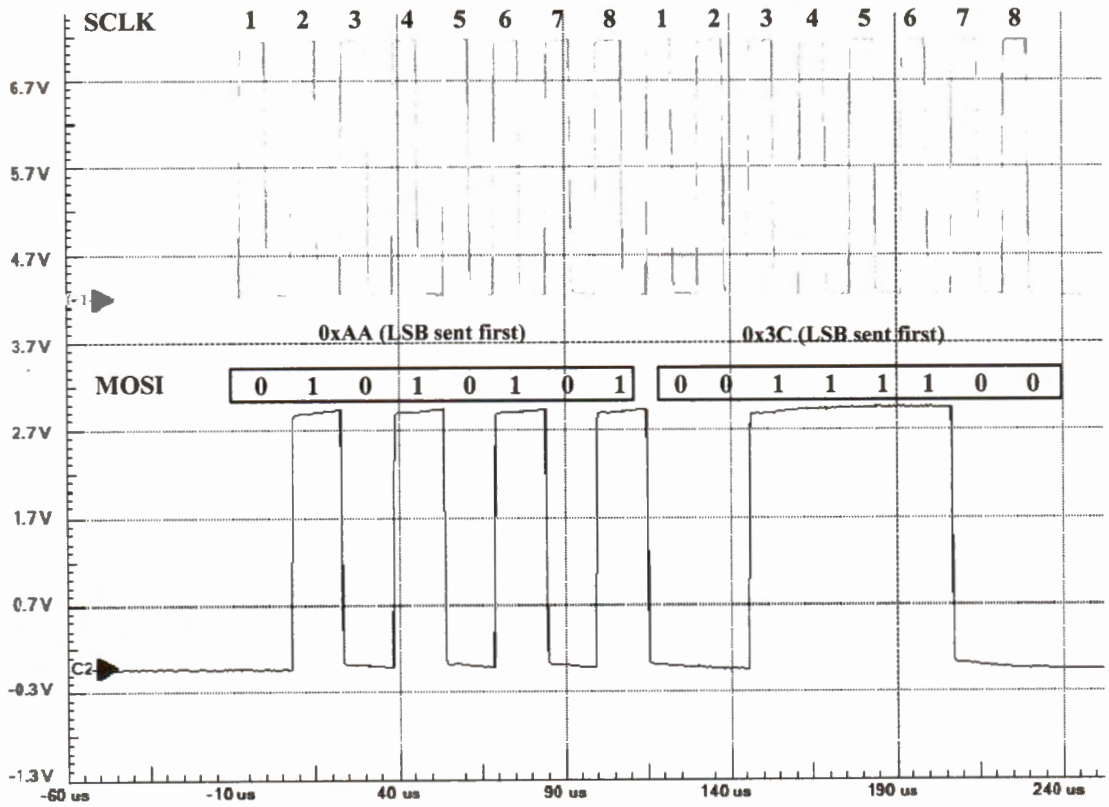


Figure 22-34. `SCLK` and `MOSI` signals when the master sends two bytes: `0xAA` and `0x3C`

22.3.2 Clock Configuration

The clock speed determines the data transfer rate. The data rate ranges from 1 to 20 megabits per second. The master can change the clock speed by programming the clock prescaler register. For STM32L processors, the baud rate control factor is stored in the `BR[2:0]` bits of the SPI control register (`CR1`). The `SCLK` clock frequency is programmed by setting the baud rate control factor.

$$f_{SCLK} = \frac{f_{SYSCLK}}{2^{1+BR[2:0]}}$$

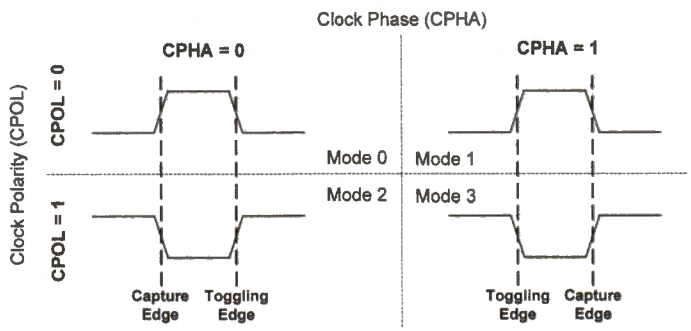


Figure 22-35. Configuration of clock phase and clock polarity

Four possible clock modes are available to program the clock edge used for data sampling and data toggling, as shown in Figure 22-35. The clock modes depend on two parameters: clock phase (`CPHA`) and clock polarity (`CPOL`). When `CPOL` is `0`, the `SCLK` line is pulled low during idle time. When `CPOL` is `1`, the `SCLK` line is pulled high during idle. When `CPHA` is `0`, the first clock transition (either rising or falling) is the first data capture edge. When `CPHA` is `1`, the second clock transition is the first capture edge.

The combination of `CPOL` and `CPHA` selects the clock edge for transmitting and capturing data. The capture of the first bit is delayed a half cycle in mode `0` and `2`.

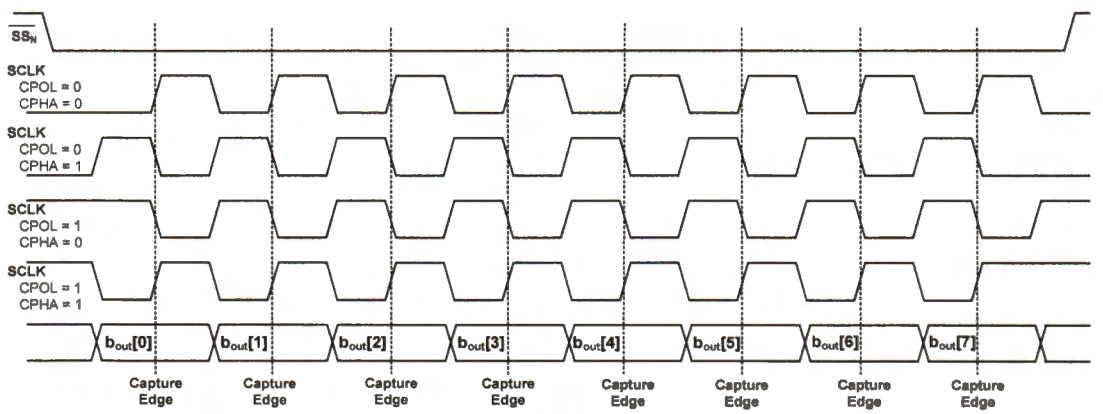


Figure 22-36. Receiving 8 bits under different settings of `CPOL` and `CPHA`

22.3.3 Using SPI to Interface a Gyroscope

This section shows how to interface the 3-axis gyro sensor L3GD20 which provides the angular velocity in three axes (yaw, pitch, and roll). The L3GD20 sensor supports two digital interfaces: I²C and SPI. When the voltage on the GYRO_CS pin is low, the SPI interface is selected. Otherwise, the I²C interface is selected.

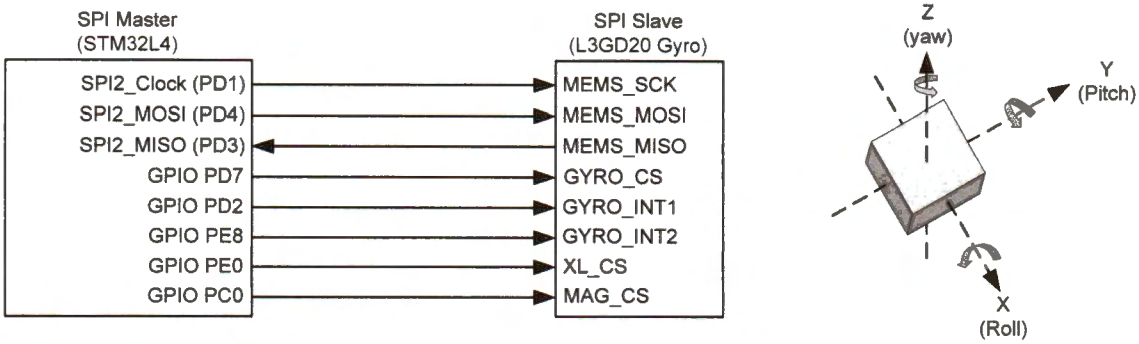


Figure 22-37. Connection between STM32L4 and L3GD20 gyroscope

The L3GD20 gyroscope internally has a set of 8-bit registers. The following program first reads the status register and checks whether angular velocity data are ready to read. If yes, the program reads 6 bytes of raw data and converts them into angular velocities.

```
#define L3GD20_STATUS_REG_ADDR          0x27  // Status register
#define L3GD20_OUT_X_L_ADDR             0x28  // Output Register
struct {
  float x;  // X axis rotation rate, degrees per second
  float y;  // Y axis rotation rate, degrees per second
  float z;  // Z axis rotation rate, degrees per second
} gyro;

int16_t gyro_x, gyro_y, gyro_z;
uint8_t gyr[6], status;

GYRO_IO_Read(&status, L3GD20_STATUS_REG_ADDR, 1); // Read status register
if ( (status & 0x08) == 0x08 ) {  // ZYXDA ready bit set
  // Read 6 bytes from gyro starting at L3GD20_OUT_X_L_ADDR
  GYRO_IO_Read(gyr, L3GD20_OUT_X_L_ADDR, 6);
  // Assume little endian (check the control register 4 of gyro)
  gyro_x = (int16_t) ((uint16_t) (gyr[1]<<8) + gyr[0]);
  gyro_y = (int16_t) ((uint16_t) (gyr[3]<<8) + gyr[2]);
  gyro_z = (int16_t) ((uint16_t) (gyr[5]<<8) + gyr[4]);
  // For +/-2000dps, 1 unit equals to 70 millidegrees per second
  gyro.x = (float) gyro_x * 0.070f;  // X angular velocity
  gyro.y = (float) gyro_y * 0.070f;  // Y angular velocity
  gyro.z = (float) gyro_z * 0.070f;  // Z angular velocity
}
```

Example 22-23. Reading x, y, and z rotation rates from the gyro sensor

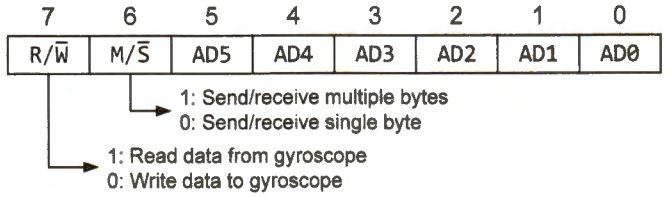


Figure 22-38. Command and address byte of internal registers in L3GD20 gyroscope

Example 22-24 and Example 22-25 show the procedure of writing data to and reading data from the gyro sensor by using the SPI interface. Figure 22-38 illustrates the bit definitions of the command and address byte. Bit 7 indicates the data transmission direction. Bit 6 shows whether single or multiple bytes will be transmitted. When the $M/\bar{S}$ bar is set, the address will be automatically incremented by 1 after each byte is transmitted.

```
// PD7: GYRO_CS (High = I2C, Low = SPI)
#define L3GD20_CS_LOW      GPIOD->ODR &= ~(1U << 7);
#define L3GD20_CS_HIGH     GPIOD->ODR |=  (1U << 7);

void GYRO_IO_Write (uint8_t *pBuffer, uint8_t WriteAddr, uint8_t size) {

  uint8_t rxBuffer[32];

  if (NumByteToWrite > 0x01) {
    WriteAddr |= 1U << 6; // Select the mode of writing multiple-byte
  }

  // Set SPI interface
  L3GD20_CS_LOW;  // 0 = SPI, 1 = I2C
  Delay(10);      // Short delay

  // Send the address of the indexed register
  SPI_Write(SPI2, &WriteAddr, rxBuffer, 1);

  // Send the data that will be written into the device
  // Bit transfer order: Most significant bit first
  SPI_Write(SPI2, pBuffer, rxBuffer, size);

  // Set chip select High at the end of the transmission
  Delay(10);      // Short delay
  L3GD20_CS_HIGH; // 0 = SPI, 1 = I2C
}
```

Example 22-24. Writing data to the gyro sensor via the SPI interface

```
void GYRO_IO_Read (uint8_t *pBuffer, uint8_t ReadAddr, uint8_t size) {

  uint8_t rxBuffer[32];

  // Select read & multiple-byte mode
  uint8_t AddrByte = ReadAddr | 1U << 7 | 1U << 6;

  // Set chip select low at the start of the transmission
  L3GD20_CS_LOW; // 0 = SPI, 1 = I2C
  Delay(10);     // Short delay

  // Send the address of the indexed register
  SPI_Write(SPI2, &AddrByte, rxBuffer, 1);

  // Receive the data that will be read from the device (MSB First)
  SPI_Read(SPI2, pBuffer, size);

  // Set chip select High at the end of the transmission
  Delay(10);      // Short delay
  L3GD20_CS_HIGH; // 0 = SPI, 1 = I2C
}
```

Example 22-25. Receiving data from the gyro sensor via the SPI interface

The following shows the initialization of SPI, which sets SPI as the master.

```
void SPI_Init(SPI_TypeDef * SPIx){

  // Enable SPI clock
  if(SPIx == SPI1){
    RCC->APB2ENR   |= RCC_APB2ENR_SPI1EN;      // Enable SPI1 Clock
    RCC->APB2RSTR  |= RCC_APB2RSTR_SPI1RST;    // Reset SPI1
    RCC->APB2RSTR  &= ~RCC_APB2RSTR_SPI1RST;   // Clear the reset of SPI1
  } else if(SPIx == SPI2){
    RCC->APB1ENR1  |= RCC_APB1ENR1_SPI2EN;     // Enable SPI2 Clock
    RCC->APB1RSTR1 |= RCC_APB1RSTR1_SPI2RST;   // Reset SPI2
    RCC->APB1RSTR1 &= ~RCC_APB1RSTR1_SPI2RST;  // Clear the reset of SPI2
  } else if(SPIx == SPI3){
    RCC->APB1ENR1  |= RCC_APB1ENR1_SPI3EN;     // Enable SPI3 Clock
    RCC->APB1RSTR1 |= RCC_APB1RSTR1_SPI3RST;   // Reset SPI3
    RCC->APB1RSTR1 &= ~RCC_APB1RSTR1_SPI3RST;  // Clear the reset of SPI3
  }

  SPIx->CR1 &= ~SPI_CR1_SPE;  // Disable SPI

  // Configure duplex or receive-only
  // 0 = Full duplex (transmit and receive), 1 = Receive-only
  SPIx->CR1 &= ~SPI_CR1_RXONLY;
```

```
// Bidirectional data mode enable: This bit enables half-duplex
// communication using common single bidirectional data line.
// 0 = 2-line unidirectional data mode selected
// 1 = 1-line bidirectional data mode selected
SPIx->CR1 &= ~SPI_CR1_BIDIMODE;

// Output enable in bidirectional mode
// 0 = Output disabled (receive-only mode)
// 1 = Output enabled (transmit-only mode)
SPIx->CR1 &= ~SPI_CR1_BIDIOE;

// Data Frame Format
SPIx->CR2 &= ~SPI_CR2_DS;
SPIx->CR2  =  SPI_CR2_DS_0 | SPI_CR2_DS_1 | SPI_CR2_DS_2;  // 0111: 8-bit

// Bit order
// 0 = MSB transmitted/received first
// 1 = LSB transmitted/received first
SPIx->CR1 &= ~SPI_CR1_LSBFIRST;  // Most significant bit first

// Clock phase
// 0 = The first clock transition is the first data capture edge
// 1 = The second clock transition is the first data capture edge
SPIx->CR1 &= ~SPI_CR1_CPHA; // 1st edge

// Clock polarity
// 0 = Set CK to 0 when idle
// 1 = Set CK to 1 when idle
SPIx->CR1 &= ~SPI_CR1_CPOL; // Polarity low

// Baud rate control:
// 000 = f_PCLK/2    001 = f_PCLK/4    010 = f_PCLK/8    011 = f_PCLK/16
// 100 = f_PCLK/32   101 = f_PCLK/64   110 = f_PCLK/128  111 = f_PCLK/256
// SPI baud rate is set to 5 MHz
SPIx->CR1 |= 3U<<3;          // Set SPI clock to 80MHz/16 = 5 MHz

// CRC Polynomial
SPIx->CRCPR = 10;

// Hardware CRC calculation disabled
SPIx->CR1 &= ~SPI_CR1_CRCEN;

// Frame format: 0 = SPI Motorola mode, 1 = SPI TI mode
SPIx->CR2 &= ~SPI_CR2_FRF;

// NSSGPIO: The value of SSI is forced onto the NSS pin and the IO value
// of the NSS pin is ignored.
// 1 = Software slave management enabled
// 0 = Hardware NSS management enabled
SPIx->CR1 |= SPI_CR1_SSM;
```

```
  // Set as Master: 0 = slave, 1 = master
  SPIx->CR1 |= SPI_CR1_MSTR;

  // Manage NSS (slave selection) by using Software
  SPIx->CR1 |= SPI_CR1_SSI;

  // Enable NSS pulse management
  SPIx->CR2 |= SPI_CR2_NSSP;

  // Receive buffer not empty (RXNE)
  // The RXNE flag is set depending on the FRXTH bit value in the SPIx_CR2 register:
  // (1) If FRXTH is set, RXNE goes high and stays high until the RXFIFO level is
  //     greater or equal to 1/4 (8-bit).
  // (2) If FRXTH is cleared, RXNE goes high and stays high until the RXFIFO level is
  //     higher than or equal to 1/2 (16-bit).
  SPIx->CR2 |= SPI_CR2_FRXTH;

  // Enable SPI
  SPIx->CR1 |= SPI_CR1_SPE;
}
```

Example 22-26. Initializing SPI

The following subroutine is for the SPI master to send the data to an SPI slave.

- It checks the transmission buffer empty flag (TXE) and waits until hardware sets TXE. If TXE is set, the transmission register is ready to accept the next data to be transmitted.
- Writing to the SPI data register (DR) automatically clears the TXE flag.
- The subroutine also waits until the busy flag is cleared to ensure the last data has been successfully sent.

```
void SPI_Write(SPI_TypeDef * SPIx, uint8_t *txBuffer, uint8_t * rxBuffer, int
size) {
  int i = 0;

  for (i = 0; i < size; i++) {

    // Wait for TXE (Transmit buffer empty)
    while( (SPIx->SR & SPI_SR_TXE ) != SPI_SR_TXE );
    SPIx->DR = txBuffer[i];

    // Wait for RXNE (Receive buffer not empty)
    while( (SPIx->SR & SPI_SR_RXNE ) != SPI_SR_RXNE );
    rxBuffer[i] = SPIx->DR;

  }

  // Wait for BSY flag cleared
  while( (SPIx->SR & SPI_SR_BSY) == SPI_SR_BSY );
}
```

Example 22-27. Send data to an SPI slave by using polling

The following subroutine allows an SPI master to receive data from an SPI slave. Only the master can initiate the data transfer and controls the communication clock (SCLK). Therefore, the master must send a dummy byte data to the slave to start the clock.

```
void SPI_Read(SPI_TypeDef * SPIx, uint8_t *rxBuffer, int size) {
  int i = 0;
  for (i = 0; i < size; i++) {
    // Wait for TXE (Transmit buffer empty)
    while( (SPIx->SR & SPI_SR_TXE ) != SPI_SR_TXE );
    // The clock is controlled by master.
    // Thus, the master must send a byte
    SPIx->DR = 0xFF;  // A dummy byte

    // data to the slave to start the clock.
    while( (SPIx->SR & SPI_SR_RXNE ) != SPI_SR_RXNE );
    rxBuffer[i] = SPIx->DR;
  }

  // Wait for BSY flag cleared
  while( (SPIx->SR & SPI_SR_BSY) == SPI_SR_BSY );
}
```

Example 22-28. Receive data from an SPI slave by using polling

In Example 22-27 and Example 22-28, the SPI master uses a polling approach to send and receive data from an SPI slave. A more efficient approach is to use SPI interrupt or SPI DMA. Section 22.1.5 and 22.1.6 shows how to use interrupt and DMA for UART communication. Similarly, SPI can also use interrupt and DMA.

If enabled, a wide range of SPI events can generate interrupt requests. These events include:

1. transmit TXFIFO ready to accept new data,
2. data received in receive RXFIFO,
3. master mode fault when a bus conflict has been detected in multi-bus communication,
4. overrun error when RXFIFO is full and cannot accept new data,
5. TI frame format error when NSS signal does not follow the data format, and
6. CRC protocol error when the received CRC value does not match the CRC value calculated based on the received data.

SPI communication handled via DMA is the most efficient. Software can enable DMA by setting the RXDMAEN and TXDMAEN bit in the CR2 register. If enabled, hardware automatically generates a DMA request each time when the TXE or RXNE enable bit in the CR2 register is set. SPI also supports a special DMA mode in which hardware generates DMA requests when the receive or transmit FIFO reaches a pre-defined threshold.

22.4 Universal Serial Bus (USB)

Universal serial bus (USB) is a widely-used industry standard to connect multiple peripheral devices to a host (typically a computer). Compared to other serial or parallel communication standards, USB has the advantage of ease of use (such as plug and play, hot swapping without rebooting, and no power supply required), low cost, low power consumption, and fast data transfer.

USB `1.0`, `2.0` and `3.0` specifications were officially released in 1996, 2000, and 2008, respectively. They are backward compatible. USB `2.0` also includes the USB OTG (on-the-go) protocol, which allows a USB device to perform both the master and slave roles. For example, a printer is a USB slave to a host PC, but it can also be a master when a USB flash drive is plugged in. In this chapter, we only cover the fundamental concepts of USB `1.0` and `2.0`. This book does not cover USB OTG and USB `3.0`.

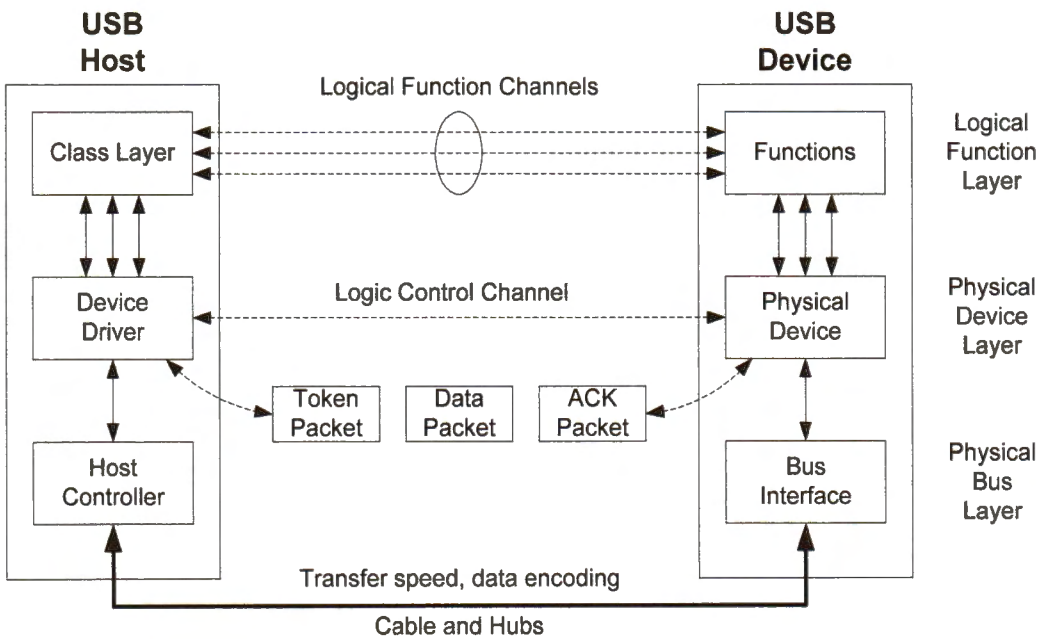


Figure 22-39. USB protocol stack

The USB protocol has three hierarchical layers.

- The bus layer takes care of wire connections, power supply to peripheral devices, transfer speed, and data signal encoding.
- The device layer establishes a logic control channel (endpoint `0`) for the host to control and set up the device, detects errors in a packet, and breaks a high-level request into multiple packets.

- The logic function layer establishes multiple logical function channels between a host and a USB device. A USB device might have multiple functions. For example, a printer may have four functions: print, photocopy, scan, and fax. A logical channel allows the host to read data from or write data to a specific function of a USB device.

22.4.1 USB Bus Layer

USB supports four transmission speeds:

- low speed (1.5 Mbit/s = 187 KB/s)
- full speed (12 Mbit/s = 1.5 MB/s)
- high speed (480 Mbit/s = 60 MB/s)
- super speed (4.8 Gbit/s = 600 MB/s)

A standard USB cable has four shielded wires: ground, Vbus (5 volts), data plus (D+) and data minus (D-). The Vbus can provide power supply to USB devices. The D+ and D- wires are physically twisted to cancel out external electromagnetic interference. Voltage under 0.3V on a wire is considered low, and voltage over 2.8V is high.

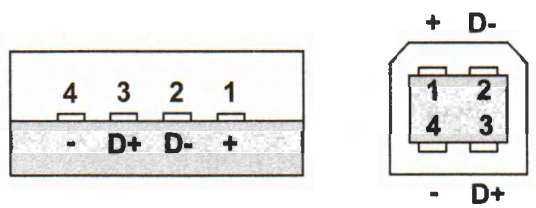


Figure 22-40. USB Connector type A plug (left) and type B plug (right)

Every high-speed USB device must support a data rate of 480Mb/s, with a clock accuracy of ±500 PPM (part per million). One PPM is 0.0001% or 1E-6. A PPM of 12 means a maximum error of approximately one second after one day has passed. 500 PPM implies the clock is off by up to 43 seconds per day. The internal clocks of a microprocessor often do not provide such high accuracy and, therefore, an external crystal oscillator is often deployed to drive the USB peripheral. For example, the internal clocks of the STM32L processor only provide an accuracy of ±600 PPM at room temperature, which implies the clocks can be up to 52 seconds off per day and up to 26 minutes off per month. An inexpensive external crystal oscillator is within ±20 PPM typically. STM32L4 can use a low-speed external clock to calibrate internal clocks.

Data are transmitted via the D+ and D- wires using differential signals. One of them must be high, and the other must be low. For example, for a full-speed connection, the wires are either in the "J" state (D+ = high and D- = low) or the "K" state (D+ = low and D- =

high). When no data is transferred (*i.e.*, idle state), the wires are in the J state. The states for low-speed are the opposite of the states for full-speed.

Non-return-to-zero inverted (NRZI) is used to encode a sequence of binary bits.

- Maintaining the current state denotes a binary `1`.
- A binary `0` is represented by switching from the J state to the K state or from the K state to the J state (also called change-on-zero).

Figure 22-41 gives an example of encoding a binary bit string.

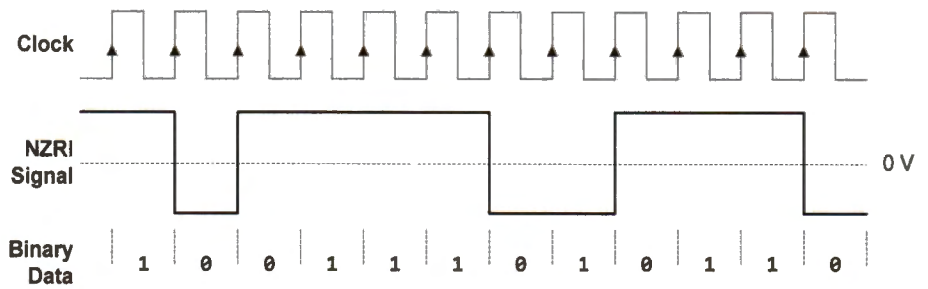


Figure 22-41. Example of NRZI data encoding for full speed

USB also uses the D+ and D- wires to transmit single-ended signals. We say the bus is in the SE0 (SE stands for single-ended) state when both are low, and in the SE1 state when both are high. SE0 is used when a transfer ends, or USB devices are disconnected or reset. SE1 is an illegal state except for battery charging.

Additionally, USB uses a technique called ***bit stuffing*** to ensure enough state transitions on D+ and D- for clock synchronization between the transmitter and the receiver. More specifically, an additional `0` bit is inserted into the bit streams after six consecutive ones.

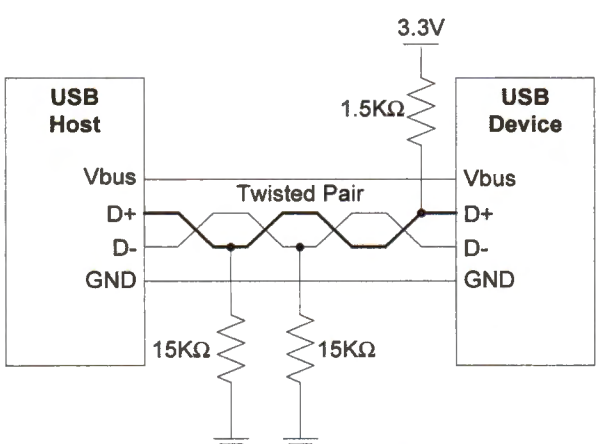


Figure 22-42. Full-speed mode (12 Mbit/s) identified by 1.5KΩ pull-up on D+

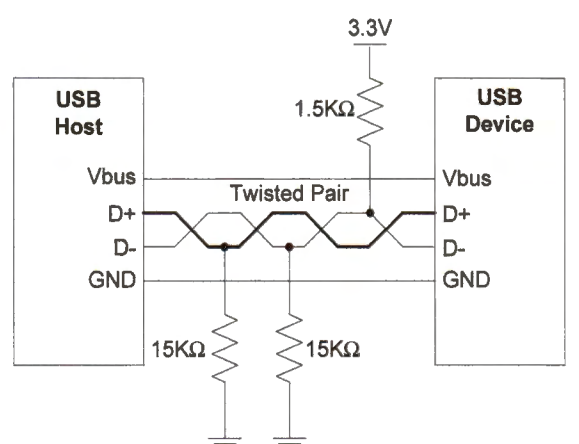


Figure 22-43. Low-speed mode (1.5 Mbit/s) identified by 1.5KΩ pull-up on D-

The USB speed is determined by pulling up the D+ or D- wire.

- When neither of them is pulled up, the host assumes no devices are connected.
- If the D+ wire is pulled up via a 1.5KΩ resistor to 3.3V, the host assumes a full-speed USB device is connected.
- The same pull-up on the D- wire indicates a low-speed device.
- A high-speed USB device is identified by initially pulling up the D+ wire. The host attempts to send or receive packets at high speed. If the communication is successful, the host assumes the device operates at high speed, and the device should remove the pull-up afterward. If the communication fails, the host assumes the device runs at full speed.

22.4.2 USB Device Layer

USB is a token-based data transfer protocol in which only a host can initiate the transfer. Each token has a target USB device address. The USB device with a matching address responds to the token packet issued by the host. The token packet also includes a target endpoint and the data transfer direction. A USB device might have multiple endpoints. Each endpoint is a predefined buffer in the USB device's memory.

The transfer direction can be either `IN` or `OUT`, from the perspective of the host.

- An `IN` token packet indicates that the host is requesting to read data from the target endpoint of the USB device.
- An `OUT` token packet indicates that the host is requesting to write data to the destination endpoint of the device.

A data transfer takes place between a host and the endpoint of a USB device. USB has four different types of data transfers: ***control***, ***bulk***, ***interrupt***, and ***isochronous***. The last three provide different tradeoffs between bandwidth, response time and reliability.

- The host uses ***control transfers*** to obtain basic information (called ***descriptors***) of a USB device and to read or set the status and address of the device. All USB devices must support control transfers.
- ***Bulk transfers*** are designed to deliver relatively large but bursty data. It provides high bandwidth but requires the application to tolerate a long delay if the bus is busy. Printers, scanners, and mass storage devices use bulk transfers.
- For ***interrupt transfers***, the host periodically queries a USB device for data. It provides less bandwidth than bulk transfers, but the maximum latency is limited to the query period. Mice and keyboards use interrupt transfer.
- ***Isochronous transfers*** provide guaranteed latency but are unreliable due to lack of error detection. Microphones and web cameras often use isochronous transfers.

At a lower level, a transfer consists of multiple packets. There are three packet formats: ***token packets***, ***data packets*** and ***acknowledge packets***. Figure 22-44 shows the packet structure of full/low speed. Each packet includes at least an SYNC byte, a PID byte, and EOP field.

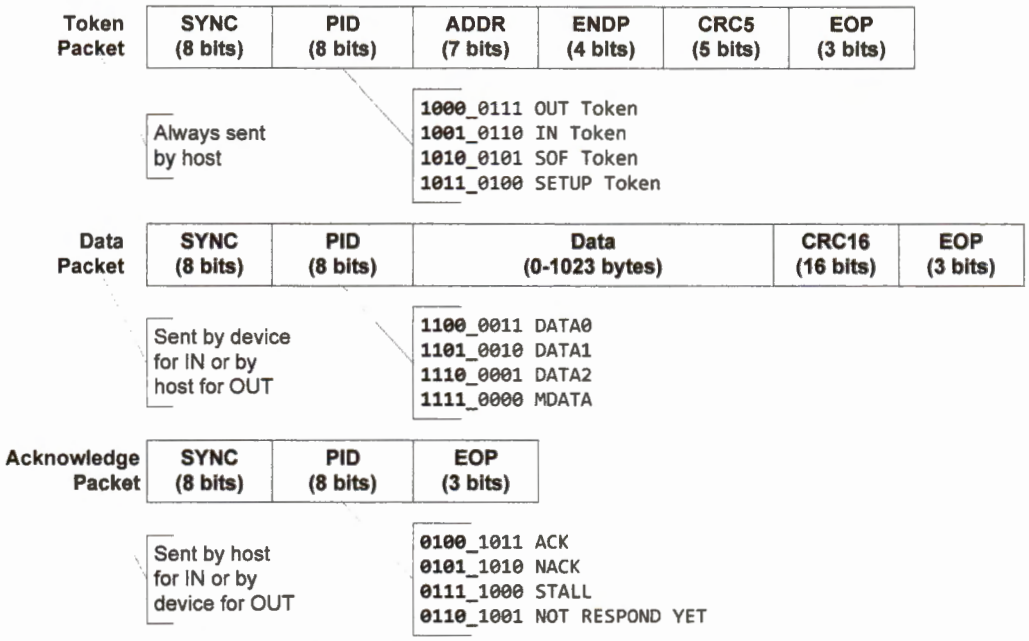


Figure 22-44. A transaction starts with a token packet sent by the host, then a data packet, and finally a handshake packet.

- **Synchronization field (SYNC).** The SYNC byte is the first byte of a packet. It is used to ensure that the receiving clock is synchronized to the transmitting clock in a packet transfer. The value of the SYNC byte is `0b00000001`. For full speed, since the idle state is the J state, the D+ and D- wires are in a sequence of "`KJKJKJKK`" when the SYNC byte is transmitted.
- **Packet identification field (PID).** The PID byte identifies the type of packet being sent. The lower four bits of the PID bytes are the inverse of the upper four bits. This inversion is used for error checking. Also, the least-significant bit is sent out first. For example, if the PID field is `0b10000111`, the actual PID code is `0b0001`.
- **Address field (ADDR).** An `ADDR` that has 7 bits can address 127 devices (address `0` is reserved). The host assigns the address. A device uses address `0` during the initial communication until the host assigns the device address.
- **Endpoint field (ENDP).** It uses four bits to identify 16 endpoints within a USB device.
- **Data field.** The length of the data field varies from 0 to 1,023, depending on the transfer type and the USB speed. For example, the data field size is limited to typically

8 bytes in low-speed devices, and to 8, 16, 32 or 64 bytes for control transfers and 64 bytes for interrupt transfer in full-speed devices.

- **Cyclic redundancy check (CRC) field**. We use CRC to detect payload corruption arising from transmission errors.
    - Each token packet has a five-bit CRC, and each data packet has a 16-bit CRC. The basic idea of the CRC calculation involves three steps. First, the data bits to be protected are treated as a binary number. Second, we divide this binary number by another predefined binary number. Finally, we select the remainder of the division as the CRC code.
    - The receiver performs the same steps to compare the remainder calculated and the rest received (*i.e.*, CRC code). Typically, we use hardware circuits to calculate CRC for fast performance.
    - We call a contiguous sequence of erroneous data bits ***burst errors***. An n-bit CRC can detect all single- and double-bit errors, and any single burst error that is shorter than or equal to n bits. It can also detect a fraction $1 - 2^{-n}$ of all longer burst errors.
- **End of packet field (EOP)**. EOP consists of SEC0 for two time units of a bit and a J-state for one time unit.

A transaction completes in three steps.

1. First, the host sends a token packet, indicating the recipient (specified in the ENDPOINT field) of the target USB device (ADDR field), and the transfer direction of the next data packet (packet ID).
2. Second, if the packet ID of the token packet is OUT or SETUP, the host sends a data packet to the specified endpoint of the target USB device, and the device must send an acknowledge packet back to the host. The acknowledge packet informs the host whether the device has successfully received the data.
3. If the token packet is an IN packet, the device sends the data packet, and the host replies with an acknowledge packet.

The USB host broadcasts a start-of-frame (SOF) packet every 1 *ms* for a full-speed bus and every 125μs for a high-speed bus. The host does not expect any USB devices to return any packet. The SOF packets provide time stamps for USB devices to schedule data transfers. For example, the host can use the SOF to inform a USB device to prepare for receiving or transmitting one data packet for an isochronous transfer.

START OF FRAME (SOF)	SYNC (8 bits)	Packet ID (8 bits)	Frame Number (11 bits)	CRC5 (5 bits)	EOP (3 bits)

Figure 22-45. Format of start-of-frame (SOF)

The USB hardware handles receiving a packet. The hardware automatically detects or generates the SYNC field, identifies the packets addressed to this USB device, performs CRC error checking, and detects or generates the end of the packet (EOP) field.

The USB hardware automatically generates interrupts for software to handle corresponding events. For example, when CRC checking fails or the device has not received any response from the host for a long time, the USB hardware generates an interrupt and sets the `ERR` bit of the USB interrupt status register.

22.4.3 USB Function Layer

A USB device may have multiple functions. For example, a web camera may have three functions: microphone, camera, and storage. A printer may have the functions of printing, scanning, and photocopying. Endpoints are the interface between a function of a USB device and the USB host. Data are transferred between the host and an endpoint. A USB device can have multiple endpoints. All USB devices must support endpoint `0`, which is a special endpoint for the host to control USB devices.

A logic communication that takes place between the host and an endpoint is called a pipe or channel. An endpoint contains the endpoint number, the transfer type (control, isochronous, bulk, or interrupt), the transfer direction (IN or OUT), the maximum packet size, and the polling intervals in terms of the number of start-of-frames.

22.4.3.1 USB Descriptors

The properties of a USB device are defined in a hierarchy of descriptors. Figure 22-46 shows example descriptors of a USB device. Each device has one and only one device descriptor. Each device can have one or more configurations. Each configuration can have multiple interfaces.

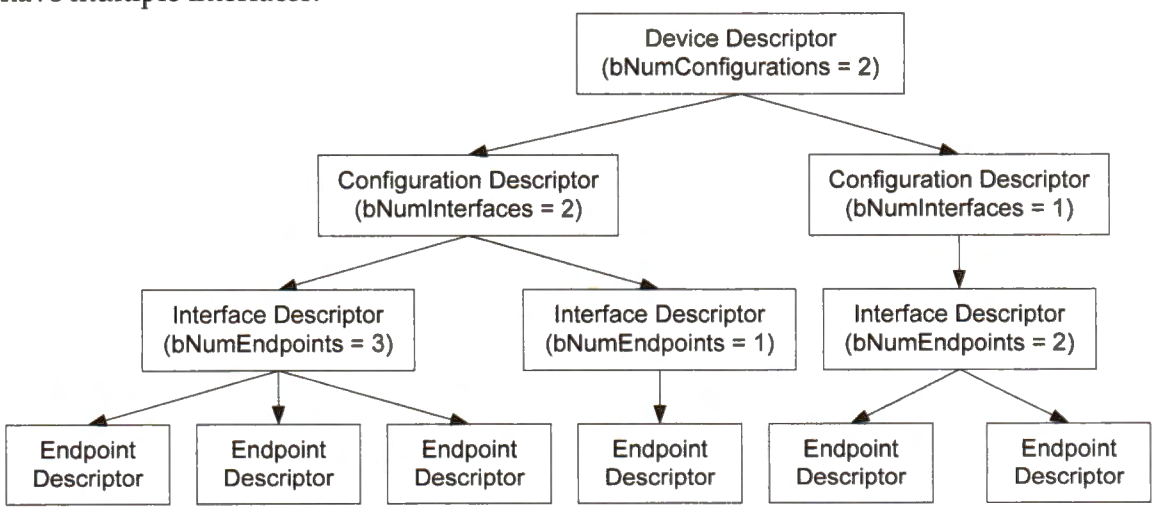


Figure 22-46. An example of hierarchical descriptors

- The device descriptor presents important information of the whole USB device, such as the vendor ID (signed by USB.org) and the product ID (assigned by the manufacturer), the total number of configurations, and the maximum packet size for endpoint `0`.
- Although it is uncommon, a device might have multiple configuration descriptors. For example, a USB device might have a configuration for USB power supply and another configuration for battery power supply. When there are multiple configuration descriptors, the USB host must select one. A configuration descriptor contains information such as the total number of interfaces used for these settings and power requirements.
- An interface descriptor is associated with a function of the USB device. A configuration can include a set of interfaces. For example, a web camera can have one interface for its microphone and another interface for its camera. An interface descriptor describes information such as the number of endpoints for this interface, and the class and subclass codes (assigned by USB.org).

Figure 22-47 shows the format of each descriptor. All descriptors have three common fields:

(1) The *length* field specifies the number of bytes in the descriptor.

(2) The *bDescriptor* field indicates the type of descriptor (`0x01` = Device, `0x02` = Configuration, `0x04` = Interface, `0x05` = Endpoint).

(3) The *bcdUSB* field states the highest USB version that the device supports in BCD code. For example, `0x0110` is USB `1.1`, `0x0200` is USB `2.0`, and `0x0300` is USB `3.0`. Chapter 18.4 discusses the BCD code.

The *bDeviceClass*, *bDeviceSubClass*, and *bDeviceProtocols* are defined by USB.org. For example, a device class code of `0x09`, `0xDC`, and `0xFF` specifies a USB hub, a diagnostic device, and a vendor specific device, respectively. When the device class code is `0x00`, the interface class code determines the device class.

The *bInterfaceClass* is also predefined by USB.org. For example, the following are some example codes of interface class: audio (`0x01`), human interface device (`0x03`), physical interface device (`0x05`), image (`0x06`), printer (`0x07`), mass storage (`0x08`), smart card (`0x0B`), content security (`0x0D`), video (`0x0E`), personal healthcare (`0x0F`), and wireless controller (`0xE0`).

There is also a string descriptor, which defines an array of strings. The *iManufacturer*, *iProduct*, *iSerialNumber*, *iConfiguration*, *iFunction* and *iInterface* used in the above descriptors are the index to the string array.

Device Descriptor

Field Name	Size	Offset
bLength	1	0
bDescriptorType	1	1
bcdUSB	2	2
bDeviceClass	1	4
bDeviceSubClass	1	5
bDeviceProtocol	1	6
bMaxPacketSize	1	7
idVendor	2	8
idProduct	2	10
bcdDevice	2	12
iManufacturer	1	14
iProduct	1	15
iSerialNumber	1	16
bNumConfigurations	1	17

Configuration Descriptor

Field Name	Size	Offset
bLength	1	0
bDescriptorType	1	1
wTotalLength	2	2
bNumInterfaces	1	4
bConfigurationValue	1	5
iConfiguration	1	6
bmAttributes	1	7
bMaxPower	1	8

Interface Descriptor

Field Name	Size	Offset
bLength	1	0
bDescriptorType	1	1
bInterfaceNumber	1	2
bAlternateSetting	1	3
bNumEndpoints	1	4
bInterfaceClass	1	5
bInterfaceSubClass	1	6
bInterfaceProtocol	1	7
iInterface	1	8

Endpoint Descriptor

Field Name	Size	Offset
bLength	1	0
bDescriptorType	1	1
bEndpointAddress	1	2
bmAttributes	1	3
wMaxPacketSize	2	4
bInterval	1	6

String Descriptor

Field Name	Size	Offset
bLength	1	0
bDescriptorType	1	1
wLANGID[0]	2	2
wLANGID[1]	2	4
wLANGID[x]	2	6

Field Name	Size	Offset
bLength	1	0
bDescriptorType	1	1
bString	n	2

Figure 22-47. Format of device, configuration, interface and endpoint descriptor

22.4.3.2 Endpoint-Oriented Communication

Each transfer over USB is identified by a three-tuple: *device address*, *endpoint*, and *direction*. Every device must support endpoint `0`, which is used for the host to control and set up the device. As shown in Figure 22-48, the web camera has four endpoints in the first configuration.

- Endpoint `0` is for the host to control the device.
- Endpoint `1` forms a channel to access the microphone.
- Endpoint `2` is to access the camera.
- Endpoint `3` is to access the storage.

To meet time constraints, isochronous transfers are preferred for transmitting audio and video signals from endpoint `1` and `2` to the host. Bulk transfers are selected for endpoint `3` IN and OUT to store and retrieve data. The web camera can have a second configuration, which includes another set of functions. The host decides which configuration is to be used for the web camera.

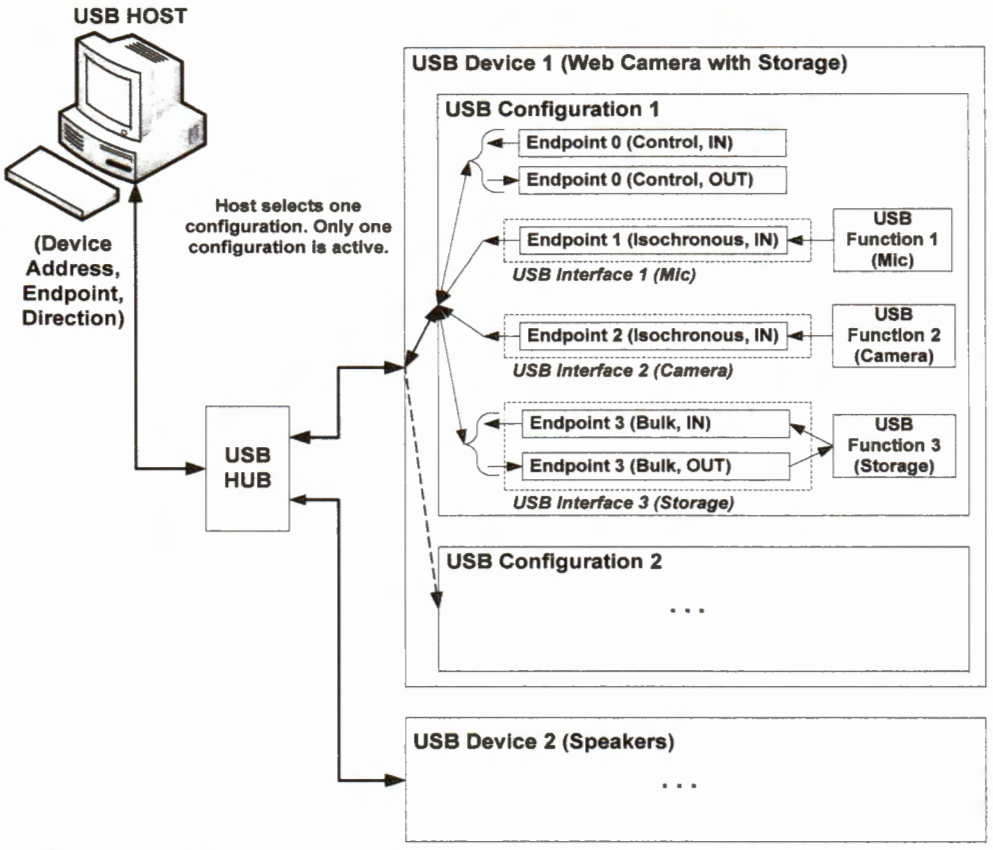


Figure 22-48. An USB configuration may have multiple functions (such as a microphone, camera, and storage device). All communications occur between the host and the endpoints of an USB configuration.

22.4.3.3 USB Enumeration

USB enumeration is the process of detecting and identifying a USB device. During a USB enumeration, the host performs the following steps:

1. Detect whether a device has been connected. When a USB device is plugged into a host, there is a change on the USB D+ or D- line because one of them is pulled up by the device.
2. Determine the USB speed. As introduced previously, pulling up the D- via 1.5KΩ pull-up to 3V indicates a low-speed device. The same pull-up on the D+ specifies a high-speed device.
3. Retrieve the device descriptor and identify what device is attached.
4. Retrieve all configuration descriptors. This process may take milliseconds to complete. The host selects one configuration.
5. Retrieve all interface descriptors.
6. Load the corresponding device driver. This is typically handled by operating systems on the host. The host uses *idVendor* and *idProduct* to match a driver.

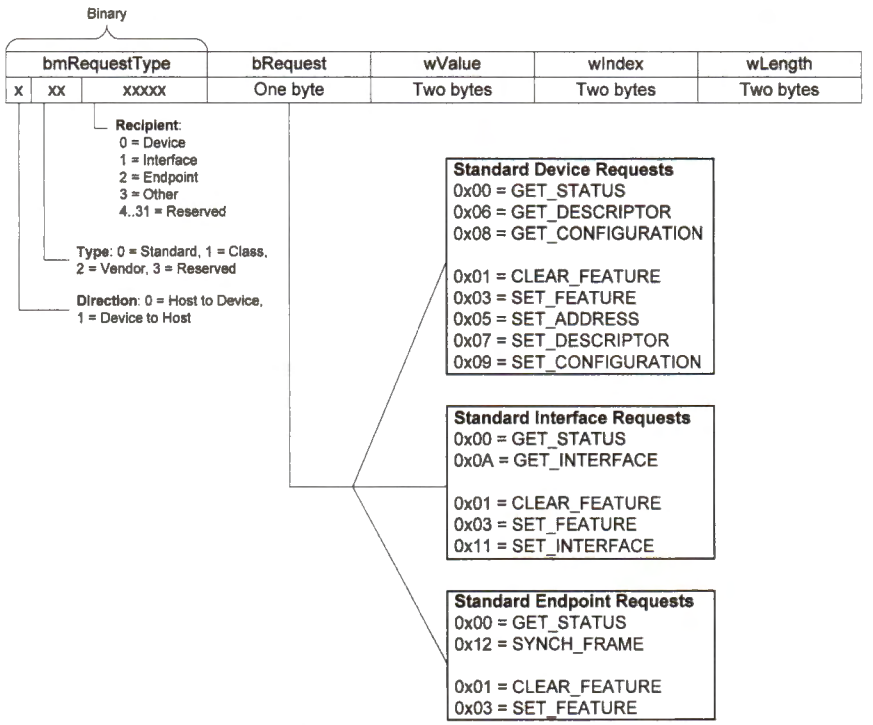


Figure 22-49. Format of setup request

The host sends a series of setup requests to complete the above enumeration process. The device responds to each setup request. Figure 22-49 shows the standardized format of a setup request. Figure 22-50 shows the procedures of retrieving the device description. Note the setup request is encapsulated into the data packet as its payload.

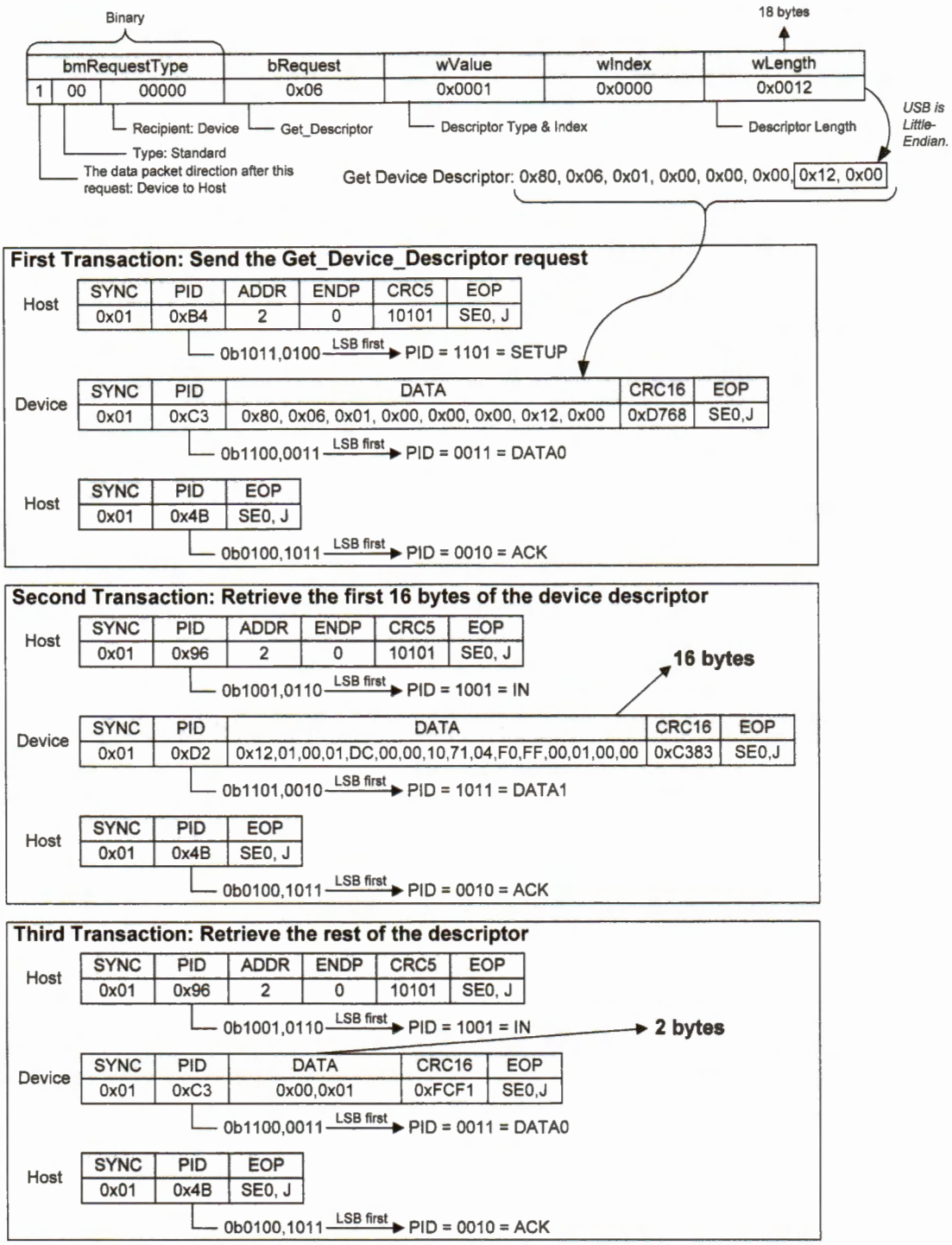


Figure 22-50. An example of sending a *get_device_descriptor* by using three transactions. Assume the maximum data size is 16 bytes. The request is sent as the payload of the first transaction.

After detecting a USB device is attached, the host waits for at least 100 *ms* to allow the completion of USB device plugging and then issues a reset request. The reset request sets the device into the default state. Initially, the default address of a USB device is `0`. Each USB device should respond to all requests addressed to `0` before it is assigned a unique address.

It is the host's responsibility to assign a unique address to the USB device. The host sends an `SET_ADDRESS` request, which includes the assigned address, to the device, as shown in Figure 22-51. The device shall respond to all requests targeted to the assigned address.

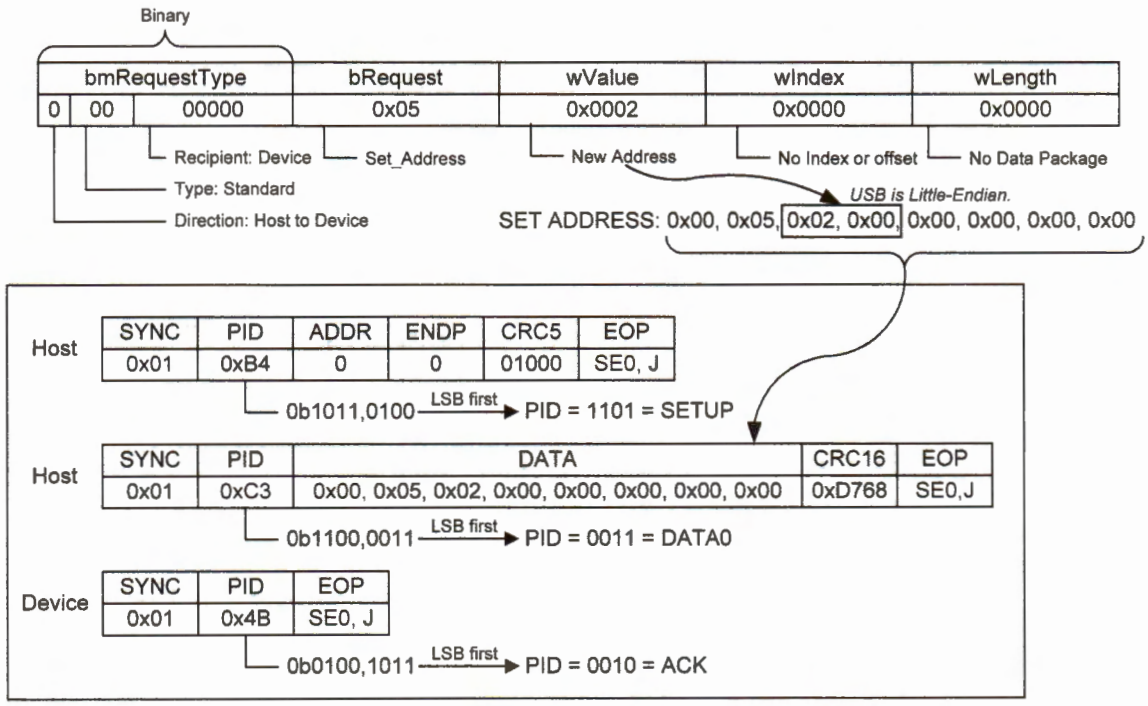


Figure 22-51. Example of sending a *set_address* request by using one transaction

Figure 22-52 shows the enumeration process in Windows operating systems.

- The host first uses a simple debouncing technique to wait for the USB device to plug in successfully and become stabilized.
- Then the host issues a reset request by using the default USB address `0`. Because the host initially does not know the packet size supported by the control endpoint (*i.e.*, endpoint `0`) of the USB device, the host issues two `GET_DESCRIPTOR` requests for the device descriptor. The first device descriptor request is to find the packet size supported by the device. Before issuing the second device descriptor request, the host sends another reset request to eliminate any confusion that the device may have. Some devices get confused if the host does not let the response to the first device descriptor request complete.

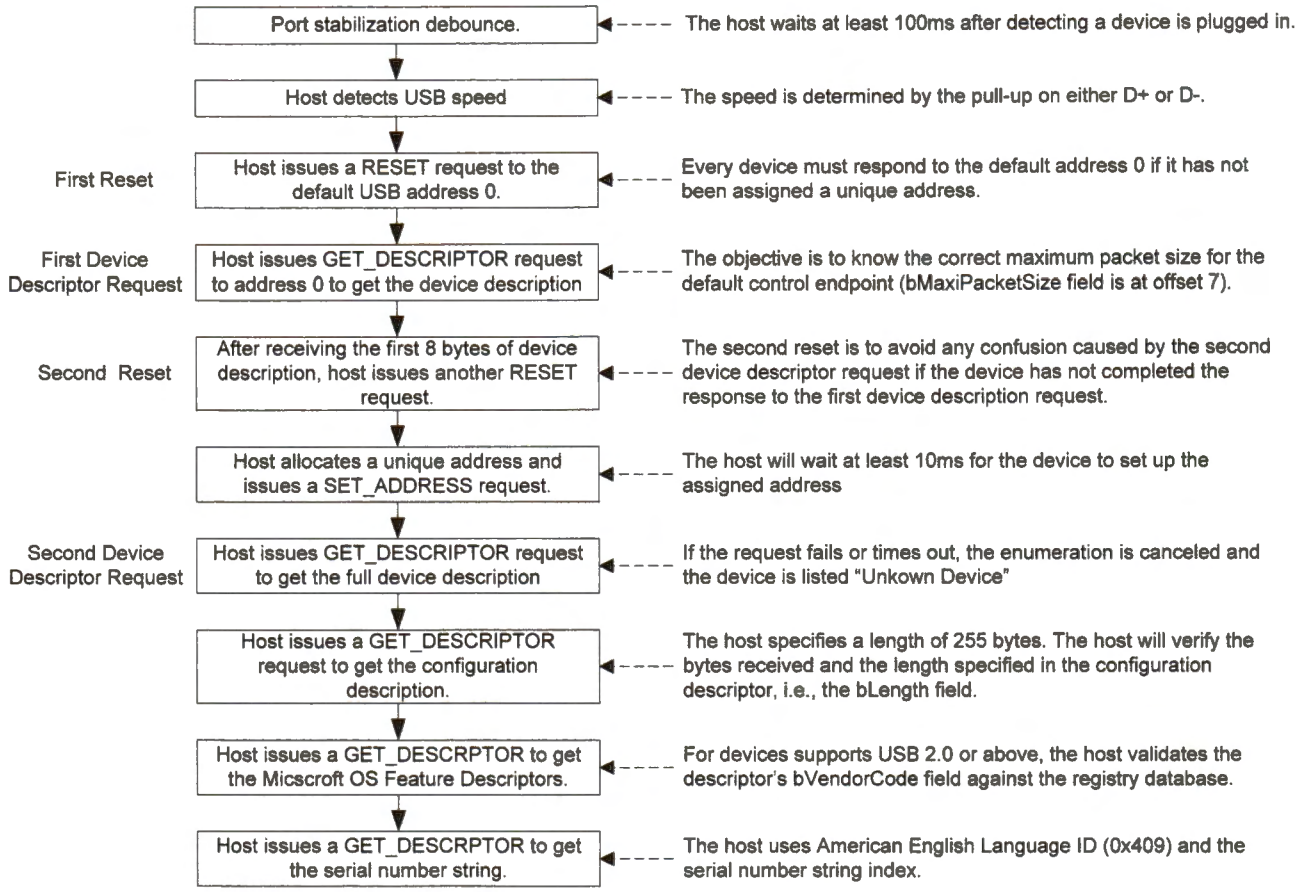


Figure 22-52. USB enumeration process in Windows

Windows operating systems (OS) also define proprietary descriptors for USB 2.0 and above, which allow the OS to install and configure the device automatically, making the process of plug and play smooth for users. The OS descriptors contain a variety of vendor-specific information, such as the identification code of a new type of device that incorporates new features of a standard USB device class or subclass. Figure 22-53 shows the format of the request that retrieves a vendor-specific OS descriptor.

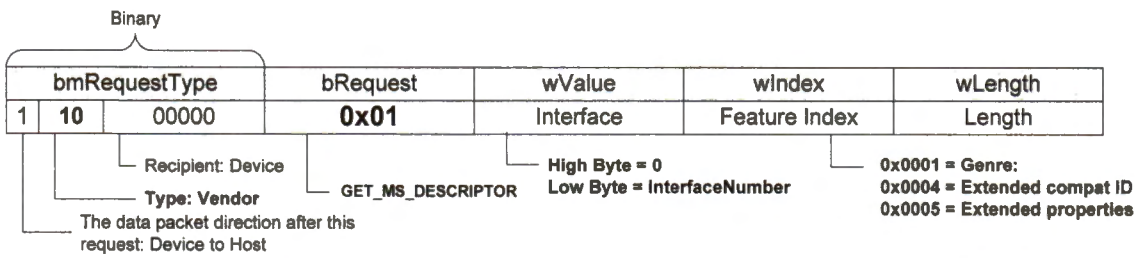


Figure 22-53. Format of the request to retrieve an OS descriptor

22.4.4 USB Class Layer

As introduced previously, a USB device can perform multiple logical functions. The device layer allows a host to send a USB request to a given function via endpoints. Each function follows a predefined class protocol to handle USB requests. Examples of standardized USB class protocols include the human interface device (HID), the communications device class (CDC), the personal healthcare device class (PHDC), the mass storage class (MSC), the audio, and the video. Vendors can also customize the USB class protocols.

The HID class specifies the interactions to human interface devices such as keyboards, mice, and game controllers. We discuss HID in this chapter later.

The CDC class emulates a virtual UART to interconnect with a serial communication port, such as RS-232 COM port. As serial ports are being gradually eliminated on personal computers, more applications utilize the CDC function of USB to communicate devices such as modems, fax machines, and telephony devices.

The PHDC class specifies the standards to interact with personal health devices such as blood pressure monitors, glucose meter, cardiovascular fitness monitor, and weight scales. The protocols are grouped into three themes: health and wellness, disease management, and aging independently. Because low latency and high reliability are very critical in some applications, this class defines meta-data along with the message data so that the host can determine how to transfer data over USB to meet the latency and reliability requirements.

The MSC class is a protocol to access a USB storage device. Modern USB storage devices use bulk transfers to achieve high bandwidth. Another important specification is the boot ability, which allows a computer to boot from an external USB storage device, instead of an internal hard drive.

The audio class uses isochronous data transfers to stream audio data at a constant rate. For full-speed USB devices, a data frame spans 1 *ms*, and a device can transfer 0-1023 bytes per data frame, depending on the application's need. As introduced previously, isochronous transfers have no acknowledge packet and no error-checking ability. Thus, a transmission error may occur.

The video class provides the function of streaming video in real time like web cameras. Like the audio class, the video class also uses isochronous transfers. A video device shall complete a bandwidth negotiation process with the host. The video class allows the host to determine preferred stream parameters to the device. After the device reports to the host, the maximum bandwidth usage based on given parameters, the host uses the

bandwidth information to identify alternate interfaces. An alternative interface is an interface that the device provides for replacing the default interface. For example, a video device with various resolutions provides different alternative interfaces that have different bandwidth requirements.

22.4.5 Human Interface Device (HID)

The HID class consists of devices that are used by humans to interact with a computer, such as a mouse, keyboard, touch screen, or game controller.

- One advantage of HID is that the host probably already has device drivers, and thus a programmer might not need to write any software for the host.
- One disadvantage of HID is that its bandwidth is relatively small because its maximum packet size for full speed is limited to 64 bytes. Because there is one data transfer per frame (1 *ms*), the bandwidth of HID is limited to 64KB/s.

HID is the interface descriptor, and it is not the device descriptor. HID specifies the class information.

- A class code of `0x03` in the interface descriptor indicates that the device is a HID device.
- ALL HID devices must have a control endpoint (endpoint `0`), an interrupt IN endpoint, and an optional interrupt OUT endpoint.
- The device data, such letters pressed on a keyboard, are sent to the host via the interrupt IN endpoint.

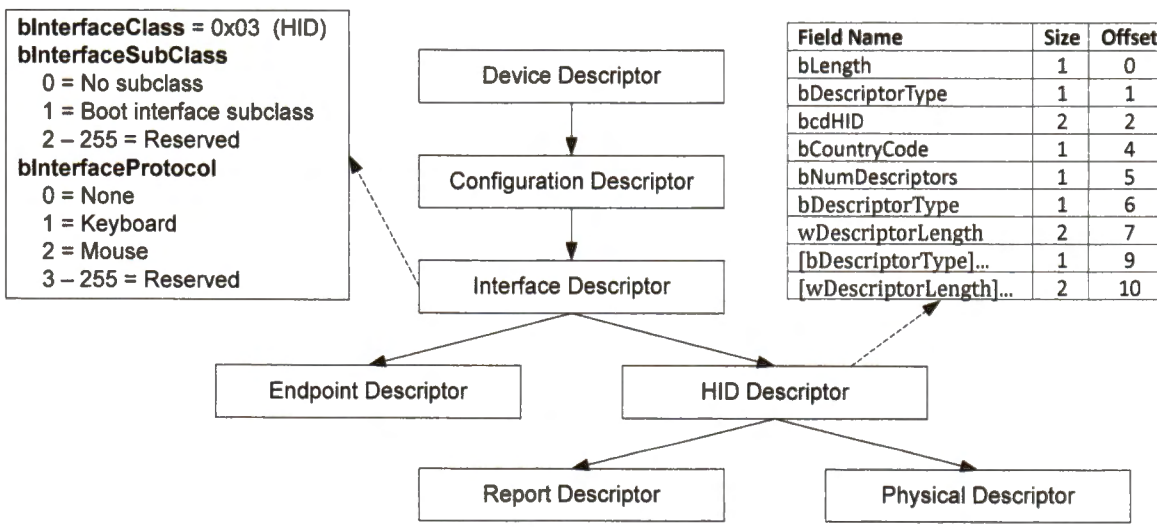


Figure 22-54. HID Descriptor

A special descriptor called ***HID descriptor*** defines the format of data exchanged between the host and the USB device.

- Example input data include the pressed key on a keyboard, and the X and Y data from a mouse.
- Example output data include LEDs indicating power status, caps lock or the number lock on a keyboard.

The host requests the HID descriptor during the USB enumeration process. A HID descriptor can include both report and physical descriptors.

- A report descriptor specifies the structure (data size, data type, and data meaning) of all data items that a device generates.
- A physical descriptor is optional and describes the part or parts of the human body used to activate the controls.

The objective of both descriptors is to help the USB host parse received data. This section gives two example HID descriptors (a keyboard and a mouse).

The following gives an example report descriptor of a keyboard. The ":" sign is to declare a bit-field in a structure. The LED structure includes three padding bits to extend the size of the structure to one byte.

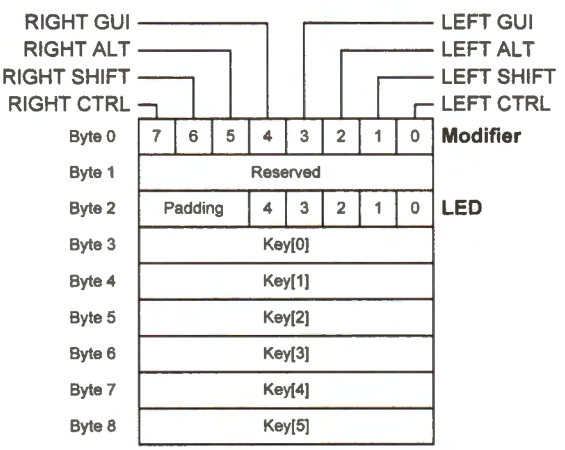


Figure 22-55. Report format of a keyboard

```
typedef struct _HID_KEYBOARD_REPORT{
  uint8_t modifier;
  // bit flags for
  // ALT, SHIFT, CTRL and GUI
  uint8_t reserved;
  struct {
      unsigned Num_Lock : 1;
      unsigned Caps_Lock : 1;
      unsigned Scroll_Lock : 1;
      unsigned Shift_Lock : 1;
      unsigned Power : 1
      unsigned Padding :3;
  } LED;
  uint8_t key[6];
} HID_KEYBOARD_REPORT;
```

HID class-specific requests are used during the enumeration. Supported class-specific requests for HID devices include `GET_REPORT`, `SET_REPORT`, `GET_IDLE`, `SET_IDLE`, `GET_PROTOCOL`, and `SET_PROTOCOL`. Figure 22-56 shows an example of `GET_REPORT` request to retrieve HID report descriptor.

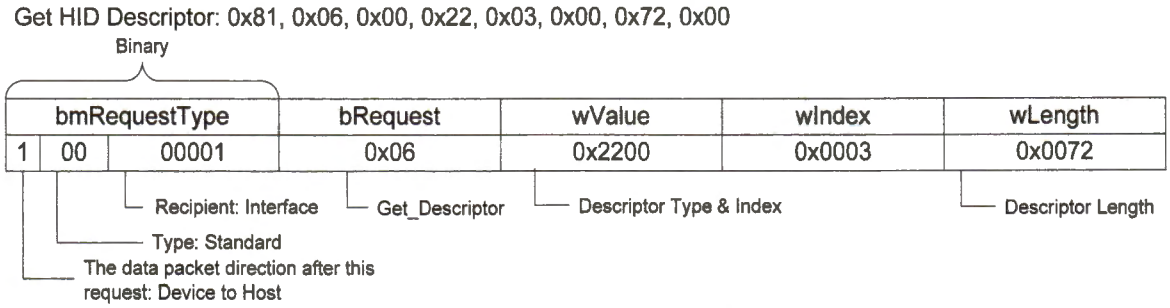


Figure 22-56. A *get_report* request to retrieve HID report descriptor

```
const uint8_t HID_Keyboard_ReportDescriptor[] = {
    0x05, 0x01,    // Usage page (generic desktop)
    0x09, 0x06,    // Usage (keyboard)
    0xA1, 0x01,    // Collection (application)
    0x75, 0x01,    //   Report size (1 bit)
    0x95, 0x08,    //   Report count (8): for 8 modifier bits
    0x05, 0x07,    //   Usage page (key codes)
    0x19, 0xE0,    //   Usage minimum (keyboard left control)
    0x29, 0xE7,    //   Usage maximum (keyboard right GUI)
    0x15, 0x00,    //   Logical minimum (0)
    0x25, 0x01,    //   Logical maximum (1)
    0x81, 0x02,    //   Input (data, variable, absolute)
    0x95, 0x01,    //   Report count (1): for reserved byte
    0x75, 0x08,    //   Report size (8 bits)
    0x81, 0x03,    //   Input (const, variable, absolute)
    0x95, 0x05,    //   Report count (5), for 5 LED outputs from the host
    0x75, 0x01,    //   Report size (1 bit)
    0x05, 0x08,    //   Usage page (LEDs)
    0x19, 0x01,    //   Usage minimum (number lock)
    0x29, 0x05,    //   Usage maximum (kana)
    0x91, 0x02,    //   Output (data, variable, absolute)
    0x95, 0x01,    //   Report count (1): LED report padding
    0x75, 0x03,    //   Report size (3 bits)
    0x91, 0x03,    //   Output (const, variable, absolute)
    0x95, 0x06,    //   Report count (6): Key arrays (6 bytes)
    0x75, 0x08,    //   Report size (8 bits)
    0x15, 0x00,    //   Logical minimum (0)
    0x26, 231, 0,  //   Logical maximum (231)
    0x05, 0x07,    //   Usage page (keyboard)
    0x19, 0x00,    //   Usage minimum (reserved)
    0x29, 231,     //   Usage maximum (keyboard application)
    0x81, 0x00,    //   Input (data, array, absolute)
    0xC0           // End collection
};
```

Table 22-10. HID keyboard report descriptor

Each row item in the HID descriptor includes a predefined type and value. For example, the second row in the HID keyboard report descriptor given in Table 22-10 has a type-value pair "0x09, 0x06",

- 0x09 represents the data type, and
- 0x06 represents the value.

The type 0x09 means the upper byte of the usage definition and the value 0x06 represents keyboard or keypad. The values are predefined, such as 0x02 for mouse, 0x0B for telephony devices, 0x0D for digitizers, and 0x80 for monitor devices.

The HID descriptor defines a report data structure with a total of eight bytes, as shown in Figure 22-55. The first byte of the data structure is a bitmap, which includes eight logical flags. The following two lines define the report size and the report count.

```
0x75, 0x01,    // Report size (1 bit)
0x95, 0x08,    // Report count (8)
```

The second byte is a reserved byte. The third byte defines five LED outputs from the USB host and three unused padding bits.

```
0x95, 0x05,    // Report count (5), for 5 LED outputs from the host
0x75, 0x01,    // Report size (1 bit)
...
0x95, 0x01,    // Report count (1): LED report padding
0x75, 0x03,    // Report size (3 bits)
```

Following the LED byte is a byte array, which is used to hold six key values. Note a keyboard does not send an ASCII value to the host when a key is pressed. Instead, it sends the HID key code, as shown in Appendix H.

Note it is often that two key inputs share the same key code. For example, the key code of "a" and "A" is 0x04. They are differentiated by the modifier byte shown in Figure 22-55. For example, when Key[0] = 0x04, it represents "A" if the LEFT SHIFT or the RIGHT SHIFT bit is set in the modifier. It represents "a" if none of these two bits is set.

The device needs to inform when a key is released. This can be done in two different forms.

- The first one is that the device sends a report in which all key values are zero.
- The second one is that the device sends a report in which different key values are stored, implying that previous keys have been released, and new keys are pressed.

The following gives an example report of a mouse with three buttons and a wheel. To create a mouse "click," two reports are needed. One is to report the button down (set the

corresponding bit in the first byte), and the other is to report the button release (clear the corresponding bit in the first byte).

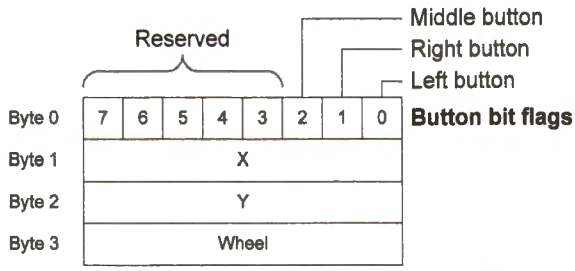


Figure 22-57. Report format of a mouse

```
typedef struct _HID_MOUSE_REPORT{
   struct {
      unsigned Left : 1;
      unsigned Right : 1;
      unsigned Middle : 1;
      unsigned Padding : 5;
    } Buttons;
    uint8_t X;
    uint8_t Y;
    uint8_t Wheel;
} HID_MOUSE_REPORT;
```

```
const uint8_t HID_ReportDescriptor[] = {
   0x05, 0x01, // Usage page (generic desktop)
   0x09, 0x02, // Usage (mouse)
   0xA1, 0x01, // Collection (application)
   0x09, 0x01, //   Usage (pointer)
   0xA1, 0x00, //   Collection (physical)
   0x05, 0x09, //     Usage page (buttons)
   0x19, 0x01, //     Usage minimum (button #1)
   0x29, 0x03, //     Usage maximum (button #3)
   0x15, 0x00, //     Logical minimum (0)
   0x25, 0x01, //     Logical maximum (1)
   0x95, 0x03, //     Report count (3), for middle, right and left buttons
   0x75, 0x01, //     Report size (1 bit)
   0x81, 0x02, //     Input (data, variable, absolute)
   0x95, 0x01, //     Report count (1)
   0x75, 0x05, //     Report size (5 bits), five padding bits
   0x81, 0x01, //     Input (const, variable, absolute)
   0x05, 0x01, //     Usage page (generic desktop)
   0x09, 0x30, //     Usage (X)
   0x09, 0x31, //     Usage (Y)
   0x09, 0x38, //     Usage (wheel)
   0x15, 0x81, //     Logical minimum (-127)
   0x25, 0x7F, //     Logical maximum (127)
   0x75, 0x08, //     Report size (8 bits)
   0x95, 0x02, //     Report count (3), for x, y, and wheel
   0x81, 0x06, //     Input (data, array, absolute)
   0xC0        //   End collection
   0xC0        // End collection
};
```

Table 22-11. HID mouse report descriptor

22.5 Exercises

1. Write an assembly program that periodically collects temperature readings from two TC74 digital temperature sensors. The sensors use I²C protocol.

2. Write an assembly program that periodically collects information from a Wii Nunchuk controller, which uses the I²C standard mode (100 Kbps). A Nunchuk has two push buttons (labeled as C and Z), an 8-bit 2-axis analog joystick (X, Y) and a 10-bit 3-axis accelerometer sensor (X, Y, and Z). It has two slave addresses, `0xA4` for writing and `0xA5` for reading. The data returned from a Nunchuk consists of 6 bytes. Note the communication is encrypted. One possible decoding is

```
Data = (Received data XOR 0x17) + 0x17
```

Address	Data							
`0x00`	Joystick X							
`0x01`	Joystick Y							
`0x02`	Accelerometer X (bit 9 to bit 2)							
`0x03`	Accelerometer Y (bit 9 to bit 2)							
`0x04`	Accelerometer Z (bit 9 to bit 2)							
`0x05`	Accel. Z (bit `1`)	Accel. Z (bit `0`)	Accel. Y (bit `1`)	Accel. Y (bit `0`)	Accel. X (bit `1`)	Accel. X (bit `0`)	C button	Z button

   The initialization command has two bytes (`0x40`, `0x00`), and a conversion command has only one byte (`0x00`). The conversion command is to ask the Nunchuk to collect the data from all its sensors and make the data ready to transfer. All commands should be sent to the slave address `0xA4`. After the conversion command, the six-byte data can read out from the slave address `0xA5`.

Joystick X	`0x80` = Center, `0x00` = Full left, `0xFF` = Full right
Joystick Y	`0x80` = Center, `0x00` = Full up, `0xFF` = Full down
Acceleration	`0 - 1023`.
Button	`0` = pressed, `1` = released

3. Use the SPI protocol to interact with 3-axis gyroscope (Parallax L3G4200D). The gyroscope provides the rate of change in rotation on its X, Y and Z axes.

4. Use the SPI protocol to read an SD memory Card.

5. Implement a HID keyboard device. When the user button on the discovery kit board is pressed, the host PC automatically plays a YouTube video.

6. Implement a HID mouse device. The device automatically draws a picture on the host screen (such as drawing a circle in Paint in Windows).